

Speech to Text Predictor



Azka Ghalib BSE223166

Zain Ahmed BSE223162

Shahzaib Arshad BSE223187

Supervised By

Dr. Rizwan Bin Faiz

Fall 2025

Department of Software Engineering

Capital University of Science & Technology, Islamabad

VERSION	V X.0
----------------	-------

NUMBER OF MEMBERS	3
-------------------	---

TITLE	Speech To Text Predictor
-------	--------------------------

SUPERVISOR NAME	Dr. Rizwan bin Faiz
------------------------	---------------------

MEMBER NAME	REG. NO.	EMAIL ADDRESS
Azka Ghalib	BSE 223166	azkawork27@gmail.com
Shahzaib Arshad	BSE223187	shahzaibarshad2610@gmail.com
Zain Ahmed	BSE223162	itxzaino2002@gmail.com

MEMBERS' SIGNATURES

Supervisor's Signature

Approval Certificate

This project, entitled as “Speech to Text Predictor” has been approved for the
award of

Bachelors of Science in Software Engineering

Committee Signatures:

Supervisor:

(Dr Rizwan Bin Faiz)

Project Coordinator:

(Mr. Ibrar Arshad)

Head of Department:

(Dr. Nadeem Anjum)

Declaration

We, hereby, declare that “No portion of the work referred to, in this project has been submitted in support of an application for another degree or qualification of this or any other university/institute or other institution of learning”. It is further declared that this undergraduate project, neither as a whole nor as a part thereof has been copied out from any sources, wherever references have been provided.

MEMBERS' SIGNATURES

Acknowledgements

We would like to express our profound gratitude to our respected supervisor, **Dr. Rizwan Bin Faiz**, for his continuous guidance, valuable feedback, and insightful suggestions throughout the preparation of this project proposal titled “**Speech to Text Predictor**” His expertise, encouragement, and keen interest in our work have been a constant source of motivation, helping us to understand the technical and research aspects of this project more deeply.

We are especially thankful for his patience, constructive criticism, and the time he dedicated to reviewing our ideas and refining our approach. His mentorship has been instrumental in clarifying our objectives, improving our technical direction, and fostering a spirit of innovation within our team. We truly appreciate his commitment and support, which have inspired us to put forth our best efforts toward the successful execution of this project.

Dedication

This is an optional section

In this section you dedicate your project to anybody that you feel motivates you for hard work and putting effort for successful life.

Executive Summary

Stuttering is a common speech disorder characterized by involuntary repetitions, prolongations, or blocks of sounds. These disruptions often cause frustration, social anxiety and misunderstandings in everyday conversation. Traditional speech therapy can improve fluency but is typically expensive and time-consuming, making it inaccessible for many. Likewise, conventional speech to text and voice assistant technologies struggle with disfluent speech: studies show that a majority of people who stutter report being cut off or misunderstood by voice assistants. Hence, there is a lack of real-time tools to support natural conversation for people who stutter. The Speech to Text Predictor addresses this gap with a mobile app that listens to a user's speech, detects stuttering patterns, and predicts the intended words on-the-fly. It uses a deep learning pipeline trained on stuttering speech data to identify disfluent segments. Whenever the speaker hesitates or stutters, the app displays the top likely word suggestions, enabling the user to continue speaking smoothly. Importantly, the system synchronizes these suggestions in real time between the speaker's device and a listener's device via Firebase Realtime Database. This ensures both parties see the same suggestions and context instantly. The result is a more empathetic and user centered communication tool. By providing gentle, intuitive support during moments of difficulty, the app reduces conversational delays and boosts confidence. Under the hood, the flutter-based app leverages on device machine learning for fast inference and secure cloud syncing, combining technical rigor with a friendly user experience.

Table of Contents

Approval Certificate.....	iii
Declaration.....	iv
Acknowledgements.....	v
Dedication.....	vi
Executive Summary.....	vii
Chapter 1.....	1
1. Introduction.....	1
1.1. Project Introduction	1
1.2. Existing Examples / Solutions	2
1.3. Business Scope.....	4
1.4. Useful Tools and Technologies	6
1.5. Project Work Break Down.....	7
1.6. Project Time Line	9
Chapter 2.....	12
2. Background Study.....	12
2.1. Literature Review.....	12
2.2. Comparison of Previous Studies	14
2.3. Research Objective	15
Chapter 3.....	16
3. Requirements Specification and Analysis	16
3.1. Epics.....	16
3.1.1. E1: Dataset Acquisition	16
3.1.2. E2 Audio Preprocessing.....	17
3.1.3. E3 Feature Extraction	17

3.1.4.	E4 ML Model Training	17
3.1.5.	E5 Stutter Detection Engine.....	17
3.1.6.	E6 Word Prediction Engine	17
3.1.7.	E7 Patient Mobile Application.....	17
3.1.8.	E8 Doctor Monitoring Interface.....	18
3.1.9.	E9: Real Time Synchronization	18
3.1.10.	E10 Deployment and System Testing.....	18
3.2.	User-stories	19
3.2.1.	E1–US1: Dataset Collection	19
3.2.2.	E1– US2: Broken & Missing Audio Detection.....	19
3.2.3.	E1–US3: Collecting Valid Audios.....	19
3.2.4.	E2–US1: Label Cleaning (Preprocessing phase)	20
3.2.5.	E2–US2: Audio Standardization.....	20
3.2.6.	E2–US3: Trimming clips from episode audios.....	20
3.2.7.	E2–US4: Adding Clips to Maximum class.....	21
3.2.8.	E2–US5: Merge and Balancing class.....	21
3.2.9.	E2–US6: Robust Exception Handling	21
3.2.10.	E2-US7: Class Balancing of Processed Dataset	22
3.2.11.	E3–US1: Fixed Length Audio Standardization	23
3.2.12.	E3–US2: Multi-Feature Extraction (215-Combo)	23
3.2.13.	E3–US3 Class-wise Feature Aggregation and Storage.....	24
3.2.14.	E4–US1: Data Loading & Scaling.....	24
3.2.15.	E4–US2: Build CNN Model with Overfitting Prevention.....	25
3.2.16.	E4–US3: Hyperparameter Tuning	25
3.2.17.	E5–US1: Continuous Audio Monitoring	26

3.2.18.	E5–US2: Stutter Identification.....	26
3.2.19.	E6–US1: Real-Time Prediction Trigger	26
3.2.20.	E6–US2: Context aware word suggestion	27
3.2.21.	E7–US1: Real-Time Speech Interaction.....	28
3.2.22.	E7–US2: Live Prediction and Detection Display	28
3.2.23.	E8–US1: Real-Time Stutter Feed	29
3.2.24.	E8–US2: Prediction Visibility	29
3.2.25.	E9–US1: Real-Time Transcript & Stutter Synchronization	29
3.2.26.	E9–US2: Prediction & Selection Synchronization	30
3.2.27.	E9–US3: Firebase Project and App Configuration.....	31
3.2.28.	E9–US4: Live Monitor Listener Integration.....	31
3.2.29.	E10–US1: Functional Testing.....	32
3.2.30.	E10–US2: Real-Time Performance Testing	33
3.2.31.	E10–US3: Deployment Validation	33
3.3.	Test-cases.....	35
3.3.1.	Test case 1: Dataset & File Validation	35
3.3.2.	Test Case 2: Broken URL Detection	35
3.3.3.	Test case 3: Annotation Agreement Validation.....	36
3.3.4.	Testcase 4: Duplicate EP ID Check.....	37
3.3.5.	Test case 5: Sample Rate Check	38
3.3.6.	Test case 6: Clip Start and End Check.....	39
3.3.7.	Test case 7: Annotation Vote Validation	39
3.3.8.	Test case 8: Class Balancing.....	40
3.3.9.	Test case 9: Audio Error Handling	41
3.3.10.	Test case 10: Class Standardization.....	42

3.3.11. Test case 11: Audio Length Standardization	43
3.3.12. Test case 12: Feature Matrix Generation	44
3.3.13. Test case 13: Feature Saving and Management	45
3.3.14. Test case 14: Dataset Scaling	46
3.3.15. Test case 15: Model Selection and Performance Evaluation	47
3.3.16. Test case 16: Model Evaluation	48
3.3.17. Test case 17: Microphone Permission Handling	49
3.3.18. Testcase 18: Real-Time Stutter Detection	50
3.3.19. Testcase 19: Prediction Trigger and Control	51
3.3.20. Testcase 20: Word Prediction and Suggestion Handling	52
3.3.21. Testcase 21: Live Speech Session Control	53
3.3.22. Testcase 22: Prediction Display	54
3.3.23. Testcase 23: Doctor Dashboard Event Display	55
3.3.24. Testcase 24: Doctor-Patient Prediction Sync	56
3.3.25. Testcase 25: Live Monitor Synchronization	56
3.3.26. Testcase 26: Firebase Prediction Update Validation	57
3.3.27. Testcase 27: Selected Prediction Sync Validation	58
3.3.28. Testcase 28: Monitor Interface Stream	60
3.3.29. Testcase 29: Functional Testing	61
3.3.30. Testcase 30: Real-Time Performance Testing	62
3.3.31. Testcase 31: Deployment Validation	63
3.4. User Interface Design	64
3.5. Traceability Matrix	66
Chapter 4	71
4. Proposed Solution	71

4.1.	Software Architecture	71
4.2.	Data Modeling	72
4.2.1.	Relational Databases	73
4.2.2.	NoSQL Databases	74
4.3.	Workflow Diagram	77
4.4.	Third-Parties Dependencies	80
Chapter 5		81
5.	Software Development.....	81
5.1.	Coding Standards	82
5.2.	Development Environment	84
5.3.	Software Description	85
5.3.1.	Snippet 1: Patient Mobile Application Module	88
5.3.2.	Snippet 2: Real-Time Session Management Module.....	89
5.3.3.	Snippet 3 – Recording State Control	89
5.3.4.	Snippet 4 – Live Connection Indicator	90
5.3.5.	Snippet 5 (User Story: E2–US5) Audio Standardization.....	91
5.3.6.	Snippet 7 (E2–US7) Class Balancing of Processed Dataset.....	93
5.3.7.	Snippet 8 Advanced Feature Extraction & Audio Standardization (Epic E3) 94	
5.3.8.	Snippet 9 Class-wise Feature Aggregation & Storage (Epic E3)	96
5.3.9.	Snippet 10 Anti-Overfitting CNN Architecture (Epic E4)	98
5.3.10.	Snippet 11 Model Training & Evaluation (Epic E4)	100
5.3.11.	Snippet 12 – Real-Time Prediction Trigger & Suggestion Engine (E6)	100
5.3.12.	Snippet 13 – Firebase Real-Time Synchronization Module (Epic E9).....	104
5.4.	Implemented UI	107
Chapter 6		111

6. Software Deployment	111
6.1. Installation / Deployment Process Description.....	111
6.2. Machine Learning Model Training and Evaluation.....	112
6.3. Firebase Realtime Database Configuration	114
6.4. Mobile Application Build and APK Generation.....	115
6.5. Mobile Application – Patient Interface.....	116
6.6. Mobile Application – Doctor Monitor Interface.....	118
6.7. Deployment Summary	119
6.8. Current Deployment – Google Play Store	120
References.....	123
Plagiarism Report.....	124
Report Approval Certificate.....	125
Appendix.....	126

List of Figures

Figure 1-1 Work breakdown structure	9
Figure 1-2 Gant Chart	11
Figure 3.4.1 HomeScreen	61
Figure 3.4.2 Session Selection Screen	62
Figure 3.4.3 Patient UI.....	62
Figure 3.4.4 Monitor UI.....	63
Figure 4.2.1 Software Architecture Diagram.....	69
Figure 4.2.2 ERD diagram	70
Figure 4.2.3 Database Schema.....	71
Figure 4.3 Activity Diagram	74
Figure 4.3.1 Usecase Diagram	75
Figure 4.3.2 Patient SSD.....	75
Figure 4.3.3 Doctor Dashboard SSD	76
Figure 4.3.4 Sequence Diagram.....	76
Figure 5.4 Implemented UI.....	104
Figure 5.4.1 Interjection.....	104
Figure 5.4.2 Block.....	105
Figure 5.4.3 Prolongation	105
Figure 5.4.4 WordRep.....	106
Figure 5.4.5 Testing Screens.....	106
Figure 6.1: Deployment Architecture of Speech to Text Predictor System.....	112
Figure 6.2: Small CNN Model	113
Figure 6.3: Firebase Realtime Database	114
Figure 6.4: Android Studio Terminal	115
Figure 6.5: Flutter APK Output Directory – app-release.apk and app-debug.apk Files.....	116

Figure 6.6: Patient Interface – Real-Time Speech Interaction Screen Running on Samsung SM-A325F (Android Studio)	117
Figure 6.7: Doctor Monitor Interface – Real-Time Session View with Stutter Detection and Prediction Sync.....	118
Figure 6.81- Google Playstore Console	12
Figure 6.82- Google Playstore Account Verification.....	122
Figure 6.83- Friendly App name.....	123
Figure 6.84- App name on App store.....	123

LIST OF TABLES

Table 1-1 Lean Canvas Template	5
Table 1-6 Project Timeline	9
Table 2-1 Comparison with related studies	14
Table 3-1 Dataset & File Validation.....	34
Table 3-2 Broken URL Detection	34
Table 3-3 Annotation Agreement Validation.....	35
Table 3-4 Duplicate EP ID Check.....	36
Table 3-5 Sample Rate Check	37
Table 3-6 Clip Start and End Check	38
Table 3-7 Annotation Vote Validation.....	38
Table 3-8 Class Balancing.....	39
Table 3-9 Audio Error Handling	40
Table 3-10 Class Standardization	41
Table 3-11 Audio Length Standardization.....	42
Table 3-12 Feature Matrix Generation	43
Table 3-13 Feature Saving and Management	44
Table 3-14 Dataset Scaling.....	45
Table 3-15 Model Selection and Performance Evaluation.....	46
Table 3-16 Model Evaluation	47
Table 3-17 Microphone Permission Handling.....	47
Table 3-18 Real-Time Stutter Detection	48
Table 3-19 Prediction Trigger and Control.....	49

Table 3-20 Word Prediction and Suggestion Handling	50
Table 3-21 Live Speech Session Control.....	51
Table 3-22 Prediction Display	52
Table 3-23 Doctor Dashboard Event Display	53
Table 3-24 Doctor-Patient Prediction Sync.....	54
Table 3-25 Live Monitor Synchronization	54
Table 3-26 Firebase Prediction Update Validation	55
Table 3-27 Selected Prediction Sync Validation.....	56
Table 3-28 Monitor Interface Stream	58
Table 3-29 Real-Time Monitor UI Update	58
Table 3-30 Inference Latency Validation	59
Table 3-31 App Deployment Verification	60
Table 3-5 Tracebility Matrix	63
Table 5-1 Patient App Initialization	86
Table 5-2 Session Creation	86
Table 3-5 Table 5-3 Recording Toggle	87
T able 5-4 Monitoring View	88
Table 5-5 Audio Standardization	89
Table 5-6 Class Balancing.....	91
Table 5-8 Speech Feature Extraction	93
Table 5-9 Class-wise Feature Aggregations.....	95

Table 5-10 Anti overfitting	97
Table 5-11 Model Training	98
Table 5-12 Model Prediction	100
Table 5-13 Firebase Synchronization	103
Table 6.1: Deployment Status Summary – Speech to Text Predictor System.....	119

This page is kept blank

Chapter 1

1. Introduction

Communication is one of the most natural human needs, yet for people who stutter, everyday interactions can be stressful and difficult. Stuttering, also known as stammering, is a speech disorder characterized by interruptions such as repetitions, prolongations, or sudden blocks that disturb the natural flow of speech. These disfluencies often cause frustration, embarrassment, and a loss of confidence, affecting academic, professional, and social life. Although speech therapy can improve fluency, it is usually expensive, time-consuming, and not accessible to everyone. Moreover, existing digital tools like voice assistants or transcription apps are designed for fluent speech and often fail to process stuttered input correctly.

Building on the 2025 & 2024 research [1] [2], this project aims to transform academic findings into a practical, real-time solution. The **Speech to Text Predictor** focuses on making communication smoother for individuals who stutter by detecting disfluencies, converting speech into text, and providing instant visual feedback. By integrating artificial intelligence, speech recognition, and mobile technology, this project bridges the gap between clinical research and real-world communication. It aims not just to recognize speech but to support speakers emotionally and socially, promoting confidence and inclusion in everyday conversations

1.1. Project Introduction

The Speech-to-Text Predictor is an Android-based assistive mobile application developed to detect stuttering in speech and provide real-time predictive text support to improve communication. Stuttering is a speech disorder that disrupts the natural flow of speech through repetitions, prolongations, and involuntary pauses, which often makes it difficult for individuals to express their intended words clearly. Existing speech-to-text systems mainly focus on converting speech into text but do not address speech disfluency or provide assistance during real-time communication. This project aims to overcome this limitation by integrating automatic speech recognition, machine learning based stuttering detection, and predictive word

generation into a unified mobile platform. The system processes speech in real time, identifies disfluency patterns, predicts intended words, and synchronizes results between multiple users through cloud-based services. The proposed solution is inspired by recent research studies on automated stuttering detection and speech analysis [1][2], which are extended in this project to develop a practical, interactive, and real-time mobile application.

Introduction of the Beneficiary of the Project

The primary beneficiaries of the Speech-to-Text Predictor are individuals who stutter and require support in verbal communication. The application assists these users by providing real-time speech to text output and predictive word suggestions, enabling them to convey their intended speech more effectively.

Secondary beneficiaries include speech therapists, medical professionals, and educators who work with individuals affected by speech disorders. Through real-time synchronization and visual feedback, these professionals can monitor speech patterns, analyze stuttering behavior, and support therapy and learning processes more efficiently. Additionally, the system can be utilized in educational and clinical environments as a digital tool for speech analysis and communication support, making it a valuable technological solution for both individual users and professional stakeholders.

1.2. Existing Examples / Solutions

This section presents a survey of existing systems and products developed to address the problem of stuttering and speech disfluency. The review covers the historical background, working mechanisms, and limitations of prominent solutions in the domain. Based on this analysis, the gaps in existing approaches are identified, which form the basis for the proposed Speech to Text Predictor system.

1.2.1. Speech Therapy Programs and Mobile Applications

Speech therapy has long been the primary approach for managing stuttering, relying on structured exercises and professional guidance to improve speech fluency. With the advancement of mobile technology, therapy-oriented applications such as Fluency SIS [7] have

emerged to provide digital support for speech training. These applications offer guided exercises, breathing techniques, fluency-enhancing practices, and progress-tracking features that help users improve their speech gradually over time. However, despite their usefulness in long-term training, such applications do not provide real-time assistance during live conversations. When a user encounters stuttering while speaking, these systems are unable to detect disfluency or offer immediate support, limiting their effectiveness in real-world communication scenarios.

1.2.2 Altered Auditory Feedback (AAF) Devices

Altered Auditory Feedback (AAF) systems have been explored since the mid-20th century as a technological method for reducing stuttering. Commercial devices such as SpeechEasy, introduced in the early 2000s, represent a prominent example of this approach. AAF devices function by replaying the user's voice with a slight delay or frequency shift, creating the perception of speaking in unison with another speaker, which can temporarily reduce stuttering in some individuals. Nevertheless, these devices are expensive and depend on specialized hardware, which restricts their accessibility. Moreover, they do not provide speech-to-text conversion, predictive word suggestions, or real-time communication support for listeners, and their effectiveness often varies across users and situations.

1.2.3. Voice Assistants and Speech Recognition Systems

Modern speech recognition systems such as Google Assistant, Siri, and Alexa have achieved significant progress in recent years and are widely used for general-purpose voice interaction. These systems process spoken input, convert it into text, and perform corresponding actions such as retrieving information or executing commands. However, they are primarily designed for fluent speech and are not optimized for disfluent speech patterns. Individuals who stutter frequently experience misinterpretation, interruption, or rejection of their speech input when using such systems. As a result, mainstream speech recognition tools fail to provide inclusive support for people who stutter and do not address the challenges of real-time disfluency detection or predictive assistance.

1.2.4 Summary of Gaps in Existing Solutions

The review of existing solutions shows that most current systems address only limited aspects of stuttering and fail to provide comprehensive real-time communication support. Therapy applications focus on long-term improvement, while AAF devices are costly, hardware-dependent, and lack predictive or interactive features. Similarly, modern voice assistants are not designed for disfluent speech, resulting in inaccurate recognition, and support platforms offer only emotional guidance without technological intervention. Therefore, a clear gap exists in the development of an affordable, mobile, and real-time assistive system for individuals who stutter. The proposed Speech-to-Text Predictor fills this gap by integrating automatic speech recognition, machine learning-based stutter detection, and predictive word generation within a synchronized cloud-based mobile platform.

1.3. Business Scope

From a business and research perspective, the Speech to Text Predictor represents a promising solution in the domain of AI-based assistive speech technology and digital healthcare. The project addresses a growing demand for intelligent, accessible, and real-time communication tools for individuals affected by speech disfluency. Globally, millions of individuals experience stuttering, which creates a potential market for technology driven speech assistance solutions. Current alternatives, such as traditional speech therapy, hardware-based auditory feedback devices, and generic speech recognition applications, do not provide integrated real-time stutter detection, predictive assistance, and synchronized communication features within a single mobile platform. This limitation highlights a clear opportunity for innovation in the assistive speech technology market.

The **proposed system** integrates machine learning-based stuttering detection, Automatic Speech Recognition (ASR), and cloud based real-time synchronization using Firebase. This combination enables instant speech-to-text conversion, detection of disfluency patterns, predictive word suggestions, and synchronized interaction between users. As a mobile based solution, the system is scalable, cost effective, and accessible, making it suitable for both individual and institutional use.

From a business perspective, the scope of the project extends to multiple target segments:

- **Individual users:** Providing an affordable and portable assistive tool for daily communication.
- **Speech therapists and clinics:** Offering data driven monitoring, session tracking, and analytical insights.
- **Educational and rehabilitation institutions:** Supporting research, training, and therapy programs.

With its AI driven architecture and real-time capabilities, the Speech to Text Predictor has the potential to evolve into a broader digital speech assistance ecosystem. Future expansion opportunities include premium subscription models, institutional partnerships, and iOS deployment. Therefore, the project demonstrates significant potential for technological innovation and commercial applicability in the assistive healthcare and speech technology domain.

Table 1-1 Lean Canvas Template

Problem	Solution	Unique Value Proposition	Unfair Advantage	Customer Segments
Traditional stuttering diagnosis is manual, and requires therapist supervision. Existing apps lack real-time stutter detection and bilingual support.	AI based android mobile app for speech to text, stutter detection, word prediction, and real-time synchronization using Firebase.	Real time Ai powered stutter detection with predictive speech assistance and synchronized communication.	Research driven ML models integrated with real time cloud synchronization in a mobile platform.	Individuals who stutter, speech therapists, clinics, educational institutions, and researchers.
Existing Alternatives	Key Metrics	High-Level Concept	Channels	Early Adopters

• Speech therapy sessions. • AAF devices (e.g., SpeechEasy).	Accuracy $\geq 80\%$, response time < 5 s, sync delay $< 500-700$ ms, user engagement.	An intelligent mobile assistant for real-time stutter detection and speech prediction.	• Google Play Store • Therapy clinics • Social outreaching • Research collaboration	Individuals who stutter
Cost Structure			Revenue Structure	
• Cloud services (Firebase). • App deployment and maintenance.			• Freemium model • Institutional licensing • Therapist subscriptions	

1.4. Useful Tools and Technologies

This section describes the key tools and technologies selected for the design, development, testing, and deployment of the Speech-to-Text Predictor system. Each technology is chosen based on its suitability for real-time speech processing, mobile application development, machine learning integration, and cloud-based synchronization.

1.4.1 Programming Language

The project utilizes **Dart** as the primary programming language for mobile application development through the Flutter framework. Dart is selected due to its strong type safety, high performance, and support for cross-platform application development. Flutter enables the creation of responsive and consistent user interfaces for Android devices while reducing development complexity. For machine learning and speech processing components, **Python** is used because of its extensive ecosystem of libraries optimized for audio analysis and model development. Libraries such as TensorFlow, Keras, Librosa, and Scikit-learn provide efficient tools for feature extraction, model training, and evaluation, making Python an appropriate choice for implementing the stuttering detection models.

1.4.2 Development Environment

Android Studio is used as the primary Integrated Development Environment (IDE) for Flutter based mobile application development. It provides advanced debugging tools, emulator support, and seamless integration with Flutter and Android SDKs. For preprocessing, feature extraction, machine learning model development and experimentation, Visual Studio, Google Colab are to be employed. These environments offer flexibility, rapid prototyping capabilities, and access to GPU resources, which are essential for training deep learning models on speech datasets.

1.4.3 Database Technology

The project uses Firebase Realtime Database, a cloud hosted NoSQL database, to enable real time data synchronization between the speaker and listener applications. Firebase is chosen because it supports low latency communication, automatic data synchronization, offline capabilities, and scalability. Its integration with Flutter simplifies the implementation of real time collaborative features required by the system.

1.4.4 Operating System

The application primarily supports the Android operating system, as it represents the largest share of mobile device users and ensures broad accessibility. The machine learning development environment is supported on **Windows** based systems, which are commonly used for model training and development. Additionally, Flutter's cross platform capabilities allow potential future extension to iOS platforms.

1.4.5 Network Protocol

The system implements standard **HTTP/HTTPS protocols** for communication between the mobile application and the machine learning inference services. RESTful APIs are used to transmit audio data and receive prediction results securely and efficiently. HTTPS is employed to ensure data confidentiality and integrity during transmission.

1.5. Project Work Break Down

To ensure efficient project execution, the tasks of the Speech-to-Text Predictor (STP) project were distributed among the team members based on their technical skills and areas of expertise.

The division of work was planned to maintain a balanced workload while addressing the technical complexity of different project components.

Azka

- UI/UX design of the mobile application
- Feature extraction and selection for machine learning models
- Contribution to model training and evaluation
- Documentation and report writing
- Participation in testing and validation
- Deployment

Zain

- Dataset collection and preprocessing
- Machine learning model training & development
- Contribution to model training and evaluation
- Participation in integration testing

Shahzaib

- Class balancing and dataset optimization
- Firebase Realtime Database design and configuration
- Real time synchronization between speaker and listener applications
- System modeling
- Deployment

Joint Responsibilities (All Members)

- Requirement analysis and literature review
- Model training and performance evaluation
- Integration testing and quality assurance
- Final documentation and presentation preparation

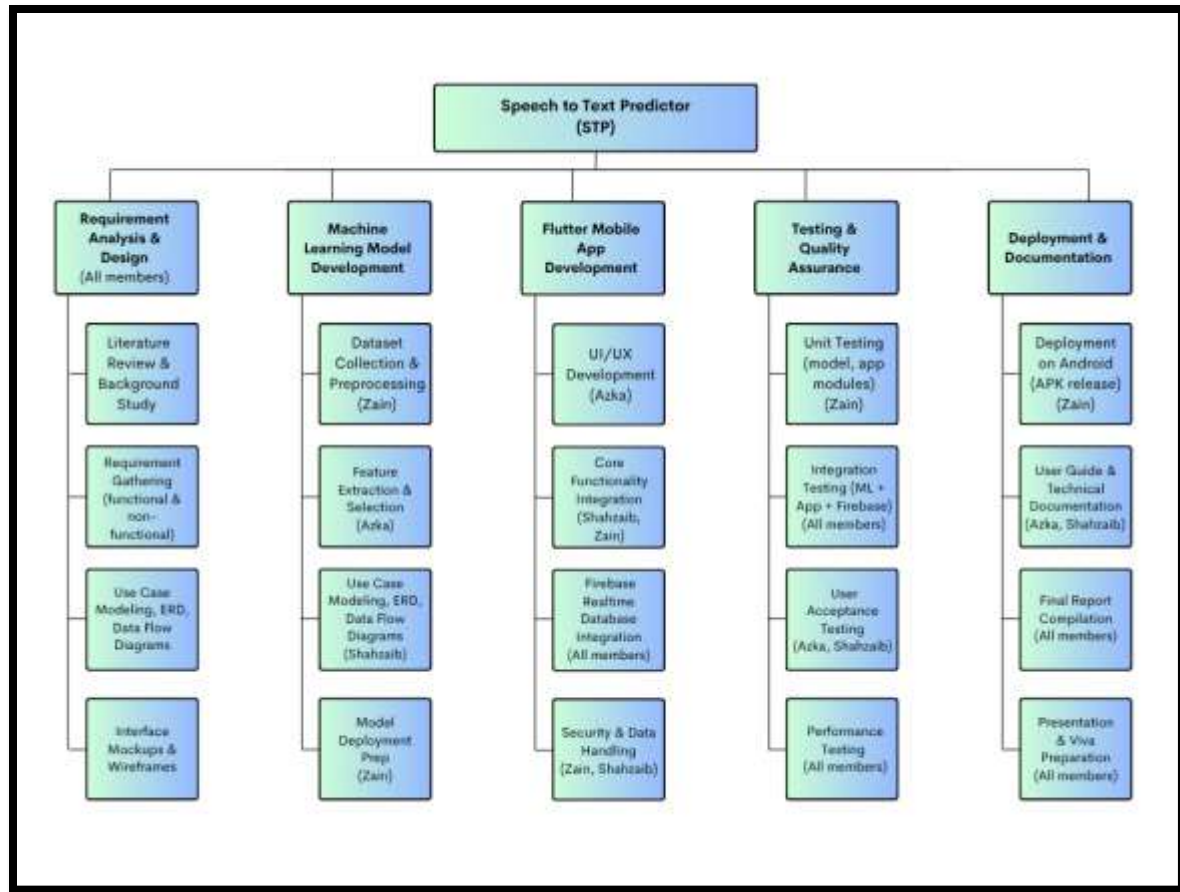


Figure 1-1 Work breakdown structure

1.6. Project Time Line

Phase	Duration (Months)	Key Activities
1. Dataset Identification & Exploratory Analysis	Month 1	- Identify and acquire relevant stuttering speech dataset (e.g., SEP-28K +fluency bank) - Conduct exploratory data analysis (EDA) to understand dataset patterns. - Review data quality, sampling rate, and disfluency distribution

2. Significant Feature Selection	Month 2-4	• Perform preprocessing (noise removal, silence, trimming) • Extract core features extraction algorithms used in previous studies. • Select significant features influencing stuttering detection
3. Model Training	Month 5-6	• Split dataset into training, validation, and test sets. Train the stutter detection model using deep learning framework • Monitor performance through accuracy and loss metrics
4. Hyperparameter Optimization	Month 7	• Tune model parameters (batch size, learning rate, layers, dropout) • Compare multiple configurations for best stability and accuracy • Optimize computation for real-time compatibility
5. MobileApp Development & Firebase Integration	Month 8 – 9	• Design and develop Flutter-based mobile interface. Integrate trained model using tflite. Establish Firebase connectivity (Auth, Firestore,) for real time sync
6. Testing & Result Analysis	Month 10 – 11	• Conduct functional, system, and usability testing. Analyze model output using confusion matrix and accuracy metrics

		• Generate performance reports and refine overall architecture
7. Result Validation & Documentation	Month 12	• Perform K-Fold cross-validation for model reliability • Validate results against test datasets • Prepare final documentation, report, and project presentation

Now, based on this explanation, our project Gantt chart is made as below,

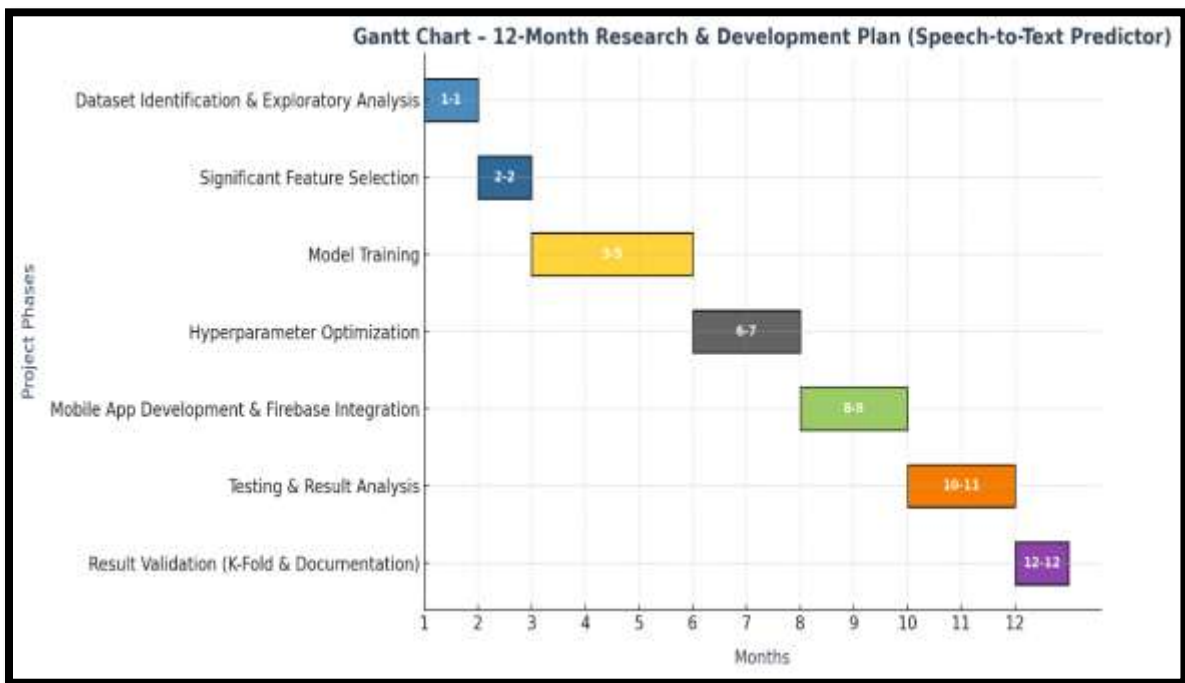


Figure 1-2 Gantt Chart

Chapter 2

2. Background Study

This chapter provides a comprehensive review of existing literature and technologies relevant to automated stuttering detection and assistive communication systems. It establishes the foundation for this project by analyzing previous work, identifying gaps, and defining the research objectives.

2.1. Literature Review

A literature review provides a systematic understanding of existing research in stuttering detection and speech-based assistive systems, helping to identify effective approaches, limitations, and research gaps. In recent years, researchers have increasingly applied machine learning and deep learning techniques to automatically detect and classify speech disfluencies such as repetitions, prolongations, interjections, and blocks [1][2].

Early studies in this domain relied on traditional machine learning models such as Support Vector Machines (SVM), Random Forest, K-Nearest Neighbors (KNN), and Decision Trees, using acoustic features extracted from speech signals. Datasets such as FluencyBank and UCLASS were widely used, but their limited size often restricted model performance and generalization ability [1]. To improve accuracy, researchers explored advanced feature extraction techniques including Mel-Frequency Cepstral Coefficients (MFCC), Zero Crossing Rate, Jitter, and Shimmer, which helped capture spectral and temporal characteristics of speech more effectively [2]. With the introduction of larger datasets such as SEP-28K [8], research shifted towards more robust and scalable models. The SEP-28K dataset contains thousands of annotated speech samples with multiple stuttering categories, enabling more accurate classification and comparative analysis of different models [1][2]. Recent studies have also employed deep learning architectures such as CNN, BiLSTM, ConvLSTM, and attention-based models, achieving significantly improved detection performance and demonstrating the effectiveness of hybrid neural networks in speech analysis [2].

Despite these advancements, existing research predominantly focuses on offline stuttering detection and classification rather than real-time assistive applications. Moreover, many studies highlight challenges such as class imbalance, limited dataset diversity, and lack of practical deployment in real world communication environments [1][2]. While some research has explored combining datasets or applying sampling techniques to address imbalance [7], few systems have been implemented as real-time mobile solutions, and none provide predictive word suggestions during live speech.

Based on this analysis, several research gaps are identified:

- 1) limited focus on real-time stuttering assistance.
- 2) insufficient mobile-based implementation of detection systems.
- 3) absence of predictive mechanisms to support users during live communication.

The proposed Speech to Text Predictor builds upon previous research by integrating automatic speech recognition, machine learning-based stutter detection, and predictive word generation into a real-time mobile application. Unlike existing approaches, the proposed system aims not only to detect stuttering events but also to assist users during communication through synchronized interaction between patient and doctor interfaces, thereby extending current research into a practical and assistive solution. Early research used small datasets such as FluencyBank and applied models like CNN, LSTM, and TDNN for stutter detection. Later studies improved performance by using advanced features such as MFCC and mel spectrograms, as well as larger datasets like SEP-28K, which enabled more robust and accurate speech analysis. However, most existing work is limited to offline stutter detection and does not address real-time deployment or practical communication support. Additionally, few studies explore mobile-based implementations, and almost no system provides live prediction of intended words during speech.

These limitations highlight the need for a real time assistive solution. Therefore, the proposed Speech to Text Predictor extends previous research by integrating speech recognition, machine learning-based stutter detection, and predictive word generation into a mobile platform, providing real-time support for individuals who stutter.

2.2. Comparison of Previous Studies

The following table provides a comparative analysis of key studies in the field of automated stuttering detection, highlighting the datasets, methodologies, and performance metrics.

Table 2-1 Comparison with related studies

Method	Reference	Proposed Strategy	Advantages	Disadvantages
FluentNet (CNN + BiLSTM)	[3]	Spectrogram based classification using attention to highlight relevant stutter patterns	High accuracy for multiple stutter types. good temporal modeling	Requires more computation. sensitive to noisy data
StutterNet (TDNN)	[4]	Time delay architecture modeling speech transitions	Fast + lightweight model suitable for basic disfluency detection	Lower performance for repeated sound stutters
CNN using SEP-28K	[5]	Trains CNN on large natural stutter dataset (eg, SEP-28K)	Better generalization due to diverse dataset	Class imbalance still affects rare stutter detection
TranStutter (Transformer)	[6]	Mel-spectrogram, transformer-based sequence modeling	Strong global pattern learning, high accuracy potential	High resource requirement, slower inference

2.3. Research Objective

The objective of this research is to design and develop a real-time speech-to-text assistive system for individuals who stutter by addressing the limitations identified in existing studies.

The specific objectives of the project are as follows:

1. To design a speech processing pipeline for detecting stuttering events in real-time speech.
2. To develop a machine learning based model capable of identifying different types of speech disfluencies.
3. To address dataset related challenges such as noise, inconsistency, and class imbalance to improve model reliability.
4. To implement a predictive mechanism that suggests the most likely intended words during stuttering.
5. To develop a android based application that provides real time speech to text assistance for users who stutter.
6. To integrate cloud-based synchronization to enable real time interaction between patient and therapist interfaces.
7. To evaluate the proposed system in terms of accuracy, latency, and usability in real world conditions.
8. To ensure scalability and flexibility of the system for future enhancements and model upgrades.

Chapter 3

3. Requirements Specification and Analysis

This chapter describes the functional and non-functional requirements of the proposed system. It defines what the system should do and how it is expected to behave under different conditions. Since requirements represent the foundation for design and development, clarity and correctness are essential. The requirements have been identified through literature review, existing research gaps, user needs, and system functionalities necessary for real-time stutter detection and speech prediction. The objective is to ensure that all system stakeholders have a clear and shared understanding of expectations before development begins.

The following sections include epics, user stories, acceptance criteria, analytical models, use case definitions and complete requirement representation from conceptual to implementation level.

3.1. Epics

Epics represent high level, large-scope functionalities of the system and each epic carries major business value. These epics will later be decomposed into detailed user stories and acceptance criteria. All epics align with the core modules of our FYP system including Dataset Module, Processing Module, Machine Learning Module, Patient App, Doctor Interface, and Backend Synchronization.

3.1.1. E1: Dataset Acquisition

Description:

As a machine learning researcher, I want to merge, organize and verify the SEP-28k & Fluency dataset so that only valid and correctly annotated audio samples are used for system training.

3.1.2. E2 Audio Preprocessing**Description:**

As a system engineer, I want to clean labels, standardize raw audio, trim clips, add clips to maximum class, merge & balance classes so that each class is trained accurately.

3.1.3. E3 Feature Extraction**Description:**

As a machine learning engineer, I want to extract structured audio features so that the model can learn speech patterns effectively.

3.1.4. E4 ML Model Training**Description:**

As a machine learning engineer, I want to train and refine the model so that it accurately detects stuttering events.

3.1.5. E5 Stutter Detection Engine**Description:**

As a system module, I want to detect stuttering moments in real time so the app can respond instantly.

3.1.6. E6 Word Prediction Engine**Description:**

As a prediction module, I want to generate top 3 intended word suggestions so the user can communicate smoothly.

3.1.7. E7 Patient Mobile Application**Description:**

As a stuttering patient, I want a simple and responsive interface so that I can speak comfortably and view predictions easily.

3.1.8. E8 Doctor Monitoring Interface**Description:**

As a doctor, I want a dashboard that shows live stutter events and predicted words so that I can monitor the patient's speech behavior.

3.1.9. E9: Real Time Synchronization**Description:**

As a backend system, I want to synchronize patient and doctor data so that both users see identical information instantly.

3.1.10. E10 Deployment and System Testing**Description:**

As a QA engineer, I want to test and validate all system modules so that the app performs smoothly in real world usage.

3.2. User-stories

3.2.1. E1–US1: Dataset Collection

Description:

As a machine learning researcher, I want to merge and organize the two datasets so that all audio clips are ready for use.

Acceptance Criteria:

- **Given** dataset sources, **When** I download the CSVs and audio clips, **then** they must be saved in a structured dataset folder.

3.2.2. E1– US2: Broken & Missing Audio Detection

Description:

As a machine learning researcher, I want to identify missing or broken audio URLs so that invalid clips do not affect preprocessing.

Acceptance Criteria:

- Given a list of audio URLs, when validation runs, then all broken or unreachable links must be flagged.
- Given missing clips, when filtering completes, then all invalid rows must be excluded from the dataset.

3.2.3. E1–US3: Collecting Valid Audios

Description:

As a machine learning researcher, I want to collect only valid audio files so that the valid audios are downloaded.

Acceptance Criteria:

- Given raw audio files, when I collect and download the CSVs & audio clips, then they must be saved in a structured dataset folder.

3.2.4. E2–US1: Label Cleaning (Preprocessing phase)

Description:

As a machine learning system developer, I want to clean and validate dataset labels by removing not downloaded episodes and duplication of episodes id number, So, that the clips are of specific episode are clipped out using the episode ID and its duration.

Acceptance Criteria:

- Given a raw dataset containing inconsistent or missing labels, When the label cleaning process is executed, Then the system shall remove not downloaded episode ID labels.

3.2.5. E2–US2: Audio Standardization

Description:

As a system engineer, I want to convert all audio files to a fixed sample rate and format so that preprocessing remains consistent.

Acceptance Criteria:

- Given mixed sample rates, when normalized, then all files must share a single sample rate (e.g., 16 kHz).

3.2.6. E2–US3: Trimming clips from episode audios

Description:

As a system engineer, I want to clip all audio files using the clip's sample in labels so that all clips can be allocated to their desired class.

Acceptance Criteria:

- Given clips duration, when processed, then clips must be trim by there specific sample.
- Given clips duration, when processed, then clips start sample must be shorter than end sample.

3.2.7. E2–US4: Adding Clips to Maximum class

Description:

As a <system engineer>, I want the clipped clips to save on class with maximum annotators votes so that the model does not train incorrectly.

Acceptance Criteria:

- Given clips labels, when processed, then clips must be identify by clip ID.
- Given clips labels, when processed, then clips must be saved in class with maximum votes.

3.2.8. E2–US5: Merge and Balancing class

Description:

As a <system engineer>, I want each class to have equal audio clips so that each class can train accurately.

Acceptance Criteria:

- Given classes, when processed, then classes with lower clips than the target must be oversampled.
- Given classes, when processed, then classes with higher clips than the target must be under sampled.

3.2.9. E2–US6: Robust Exception Handling

Description:

As a system, I want to safely handle invalid inputs and runtime exceptions during audio preprocessing so that dataset generation is reliable.

Acceptance Criteria:

- **Given** a required column is missing in the dataset, **when** preprocessing is executed, **then** the system displays an error and stops execution.
- **Given** the ClipId column is missing, **when** preprocessing starts, **then** the system automatically generates unique clip identifiers.
- **Given** an episode audio file does not exist, **when** the system processes the clip, **then** the clip is skipped and execution continues.
- **Given** the start value is greater than or equal to the stop value, **when** boundaries are validated, **then** the clip is skipped and recorded as invalid.
- **Given** an audio file is corrupted or unreadable, **when** the system attempts to read it, **then** the exception is handled and processing continues.
- **Given** the input audio is multichannel, **when** the clip is extracted, **then** it is converted to mono before saving.

3.2.10. E2-US7: Class Balancing of Processed Dataset**Description:**

As a machine learning system, I want to balance the number of audio samples across all speech classes so that the training dataset is unbiased and improves the fairness and accuracy of the prediction models.

Acceptance Criteria:

- **Given** the preprocessed and class-wise segmented audio dataset, **When** the class balancing process is executed, **then** each speech class must contain **exactly 3200 audio samples** after balancing.
- **Given** a speech class contains more than 3200 samples, **When** the balancing process runs, **Then** the system must randomly select samples to meet the target limit.
- **Given** a speech class contains fewer than 3200 samples, **When** the balancing process runs, **Then** the system must duplicate existing samples to reach the required count.

- **Given** multiple speech classes are present in the dataset, **when** class balancing is performed, **Then** the system must process all classes independently to maintain uniform class distribution.
- **Given** the class balancing process completes successfully, **When** the output dataset is reviewed, **Then** the system must ensure no data loss or corruption has occurred.

3.2.11. E3–US1: Fixed Length Audio Standardization

Description:

As a machine learning engineer, I want to load and standardize audio clips to exactly 3 seconds so that all feature arrays have a uniform and consistent shape for model training.

Acceptance Criteria:

- Given a clean 3-second audio clip, When the feature extraction process starts, then the system should extract all required audio features from the clip.
- **Given** an audio clip longer than 3 seconds, **When** processed, **then** it must be truncated to exactly 3 seconds.
- **Given** a preprocessed audio file, **When** loaded, **then** it must be loaded at a standard sampling rate of 16,000 Hz.

3.2.12. E3–US2: Multi-Feature Extraction (215-Combo)

Description:

Given a standardized 3-second audio clip, When extraction runs, then it must successfully generate 128 Mel Spectrogram, 20 MFCC, 20 MFCC Delta, 20 MFCC Delta-Delta, 12 Chroma, 1 Spectral Centroid, 1 Spectral Bandwidth, 1 Spectral Rolloff, 7 Spectral Contrast, 1 Spectral Flatness, 1 RMS, 1 ZCR, 1 Onset Envelope, and 1 Silence Mask.

Acceptance Criteria:

- Given a clean 3-second audio clip, When the feature extraction process starts, then the system should extract all required audio features from the clip.
- Given the individual extracted features, When processed, then they must be vertically

stacked into a single combined array of exactly 215 dimensions.

- Given a damaged or unreadable audio file, When the system tries to extract features, then it should skip the file safely and continue without crashing..

3.2.13. E3–US3 Class-wise Feature Aggregation and Storage

Description:

As a machine learning researcher, I want to aggregate the extracted features for each stutter class and save them as .npy arrays so that data loading during model training is highly optimized, fast, and memory-efficient.

Acceptance Criteria:

- **Given** a batch of successfully extracted features for a specific class, **When** processing completes, **then** they must be aggregated into a Numpy array and saved as <ClassName>_zcr_v2_X.npy.
- **Given** an empty class folder with no .wav files, **When** processed, **then** the system must log a warning and skip the class without creating an empty output file.
- **Given** the completion of a class batch, **When** saved, **then** the system must clear the temporary lists from memory using garbage collection to prevent memory overflow.

3.2.14. E4–US1: Data Loading & Scaling

Description:

As a machine learning engineer, I want to load extracted features, apply Min-Max scaling, and balance all classes to exactly 5000 samples so that the model trains on stable, unbiased data.

Acceptance Criteria:

- **Given** extracted. npy feature files, **when** loaded, **then** the system must apply Min-Max scaling to normalize values between 0 and 1.
- **Given** a class with more or fewer than 5000 samples, **when** processed, **then** the system must truncate or oversample (with replacement) to reach exactly 5000 samples.

- **Given** the combined dataset, **when** split, **then** it must allocate exactly 80% to training and 20% to testing.

3.2.15. E4–US2: Build CNN Model with Overfitting Prevention

Description:

As a machine learning engineer, I want to build a CNN model using Conv2D, Dense, Max Pooling, and Dropout layers so that the model can learn speech feature patterns from extracted audio maps and reduce overfitting during training.

Acceptance Criteria:

- Given the processed audio feature maps, When the CNN model is built, then it should accept input shape (215, 94, 1).
- Given the model architecture, When the layers are added, then it should include Conv2D, MaxPooling2D, Dropout, and Dense layers.
- Given the CNN model, When overfitting prevention is applied, then it should use Dropout between 0.25 and 0.40

3.2.16. E4–US3: Hyperparameter Tuning

Description:

As a machine learning researcher, I want to test different model settings so that I can choose the best settings for better accuracy.

Acceptance Criteria:

- Given the tuning process, when different learning rates, batch sizes, epochs, and dropout values are tested, then the system must select the best model settings.
- Given the best settings are selected, when the model is tested, then the system must show the final accuracy, precision, recall, and F1-score on unseen test data.

3.2.17. E5–US1: Continuous Audio Monitoring

Description:

As a <system module>, I want to continuously listen to live audio so that stutters can be captured immediately.

Acceptance Criteria:

- **Given** microphone access, **when** user speaks, **Then** the system must receive audio frames continuously.
- **Given** audio fluctuations, **when** stutter characteristics appear, **Then** the segment must be flagged.

3.2.18. E5–US2: Stutter Identification

Description:

As a <system module>, I want to classify speech segments as stutter or normal so that the app knows when the user struggled.

Acceptance Criteria:

- **Given** an audio frame, **when** processed, **Then** the system must output “stutter” or “no stutter.”
- **Given** a stutter is detected, **when** logged, **Then** the timestamp must be stored.

3.2.19. E6–US1: Real-Time Prediction Trigger

Description:

As a prediction module, I want to activate automatically when a stuttering is detected so that contextual word prediction can begin at the correct moment during live speech.

Acceptance Criteria:

- **Given** a stutter pattern is detected, **when** the trigger fires, **then** the prediction module must activate automatically.
- **Given** normal fluent speech is detected, **when** no disfluency exists, **then** prediction must not execute.
- **Given** a prediction session is already active, **when** multiple rapid triggers occur, **then** duplicate prediction requests must be prevented.

3.2.20. E6–US2: Context aware word suggestion

Description:

As a prediction module, I want to generate context-aware top word suggestions based on the user's speech prefix so that the user can quickly select the intended word and continue communication smoothly.

Acceptance Criteria:

- **Given** a valid speech prefix, **when** processed by the prediction API, **then** ranked word suggestions must be generated.
- **Given** suggestions are received, **when** displayed, **then** they must appear as tappable prediction chips in real time.
- **Given** the user selects a suggested word, **when** confirmed, **then** the blank placeholder in the transcript must be replaced with the selected word.
- **Given** the backend fails to return reliable suggestions, **when** prediction completes, **then** fallback suggestions must be displayed.
- **Given** prediction is completed, **when** the user selects a word, **then** the system must resume continuous speech listening automatically.

3.2.21. E7–US1: Real-Time Speech Interaction

Description:

As a patient, I want to interact with the microphone in real time so that I can speak naturally and continuously during a session.

Acceptance Criteria:

- **Given** the microphone button is tapped, **when** recording starts, **then** live speech monitoring must begin immediately.
- **Given** recording is active, **when** the user speaks, **then** speech must continuously appear in the transcript area.
- **Given** recording is stopped manually, **when** the session pauses, **then** live monitoring must stop safely.
- **Given** microphone permission is denied, **when** recording is requested, **then** the system must display an appropriate warning message.

3.2.22. E7–US2: Live Prediction and Detection Display

Description:

As a patient, I want to see detected stutters, prediction suggestions, and system status instantly so that I can understand and interact with the prediction system easily.

Acceptance Criteria:

- **Given** a stutter is detected, **when** prediction begins, **then** the detected stutter type must appear instantly on the interface.
- **Given** predictions are generated, **when** displayed, **then** suggestions must appear as clear and tappable prediction chips.
- **Given** a prediction word is selected, **when** transcript updates, **then** the selected word must replace the blank placeholder correctly.

- **Given** prediction selection completes, **when** listening resumes, **then** the interface status must return to “Listening...”.
- **Given** multiple prediction cycles occur continuously, **when** the session remains active, **then** the interface must remain responsive without freezing.

3.2.23. E8–US1: Real-Time Stutter Feed

Description:

As a <doctor>, I want to see stutters in real time so that I can monitor progress during sessions.

Acceptance Criteria:

- **Given** a stutter occurs on the patient’s device, **when** synced, **then** it must appear instantly on the doctor screen.
- **Given** continuous speech, **when** no stutter occurs, **Then** the feed must remain empty.

3.2.24. E8–US2: Prediction Visibility

Description:

As a <doctor>, I want to view the patient’s predicted words so that I can understand communication intent.

Acceptance Criteria:

- **Given** predictions are generated, **when** synced, **Then** the doctor must see the same 3 suggestions.
- **Given** a word is selected by the patient, **when** updated, **Then** the doctor must see the selection.

3.2.25. E9–US1: Real-Time Transcript & Stutter Synchronization

Description:

As a backend synchronization system, I want to synchronize live transcript updates, stutter detection status, and recording state using Firebase Realtime Database so that the patient and monitor interfaces display the same session information in real time.

Acceptance Criteria:

- Given the patient speaks during an active session, when the transcript updates, then the updated transcript must be stored in Firebase and displayed on the monitor screen.
- Given a stutter status is detected, when Firebase receives the updated status, then the same status must appear on both patient and monitor interfaces.
- Given recording starts or stops, when the recording state changes, then Firebase must update the state for the monitor interface.
- Given Firebase data changes, when the monitor screen is active, then it must update automatically without manual refresh.
- Given the session is active, when multiple updates occur, then synchronization must remain stable without crashing the app.

3.2.26. E9–US2: Prediction & Selection Synchronization**Description:**

As a backend synchronization system, I want to synchronize context-related prediction suggestions and selected words through Firebase Realtime Database so that both patient and monitor users can view the same prediction information during a live speech session.

Acceptance Criteria:

- Given prediction suggestions are generated, when Firebase updates the prediction list, then the same suggestions must appear on the monitor screen.
- Given the patient selects a predicted word, when the transcript is updated, then the selected word must be reflected on both interfaces.
- Given prediction suggestions are empty, when Firebase receives an empty prediction list, then the system must handle it without crashing.

- Given multiple prediction cycles occur, when Firebase updates rapidly, then the monitor screen must remain responsive.
- Given the monitor reconnects, when Firebase restores the session state, then the latest transcript and predictions must reload automatically.

3.2.27. E9–US3: Firebase Project and App Configuration

Description:

As a developer, I want to configure Firebase with the Flutter application so that the mobile app can connect to Firebase Realtime Database and use cloud synchronization services correctly.

Acceptance Criteria:

- **Given** the Firebase project is created, **when** FlutterFire CLI is configured, **then** `firebase_options.dart` must be generated successfully.
- **Given** the Android app is registered in Firebase, **when** configuration completes, **then** `google-services.json` must be added to the Android project.
- Given Firebase packages are added, when `flutter pub get` runs, then Firebase dependencies must install successfully.
- Given Firebase initialization is added in Flutter, when the app launches, then Firebase must initialize without runtime errors.
- Given Realtime Database is enabled, when the app writes session data, then the data must appear in Firebase Console.

3.2.28. E9–US4: Live Monitor Listener Integration

Description:

As a monitor interface, I want to listen to Firebase session updates continuously so that I can

display the patient's transcript, stutter status, prediction suggestions, and recording state live during the session.

Acceptance Criteria:

- Given the monitor screen is opened, when Firebase has active session data, then the latest transcript must be displayed automatically.
- Given Firebase receives new prediction suggestions, when the stream updates, then prediction chips must appear on the monitor screen.
- Given the recording state changes, when Firebase updates the value, then the monitor must show the correct live or paused state.
- Given Firebase has no data yet, when the monitor screen loads, then it must show a waiting state instead of crashing.
- Given Firebase updates continuously, when the monitor receives stream events, then the UI must refresh smoothly.

3.2.29. E10–US1: Functional Testing**Description:**

As a QA engineer, I want to test all main app modules so that stutter detection, prediction, transcription, Firebase sync, patient UI, and doctor monitor UI work correctly.

Acceptance Criteria:

- **Given** all module test cases are ready, **when** testing is performed, then all test cases should pass successfully.
- **Given** the Small CNN model is integrated, **when** stutter audio is provided, **then** the correct stutter type should be detected with at least **94.3% accuracy**.

- **Given** Firebase is connected, **when** a session is active, **then** all required session data should be updated correctly.
- **Given** a bug is found and fixed, **when** testing is repeated, **then** all affected test cases should pass before completion.

3.2.30. E10–US2: Real-Time Performance Testing

Description:

As a QA engineer, I want to check app speed so that detection, prediction, and monitor updates feel real-time during live speech.

Acceptance Criteria:

- **Given** live audio is received, **when** a stutter event is detected, **then** the system should return the detection result within **1000 ms**.
- **Given** a stutter has been detected, **when** word predictions are generated, **then** the suggested words should appear on the patient screen within **1000 ms**.
- **Given** the patient and doctor applications are connected through Firebase, **when** session data is updated, **then** the doctor monitor should reflect the changes without noticeable delay.

3.2.31. E10–US3: Deployment Validation

Description:

As a QA engineer, I want to verify that the release APK installs and executes correctly on both a physical Android device (Samsung SM-A325F, API 33) and a virtual Android emulator (Pixel 9 Pro XL), so that the application is confirmed stable and ready for production deployment.

Acceptance Criteria:

-
- **Given** the compiled app-release.apk (71.2 MB), **when** installed on Samsung SM-A325F running Android API 33, **then** the application must launch without crashes and all features must be fully functional.
 - **Given** the release APK, **when** deployed to the Pixel 9 Pro XL Android emulator in Android Studio, **then** the application must install and run identically to the physical device.
 - **Given** the release build (exit code 0, Gradle assembleRelease), **when** tested end-to-end, **then** zero crashes must be recorded across all testing sessions on both environments

3.3. Test-cases

Test Cases are defined scenarios that describe the specific inputs, execution conditions, and expected outcomes used to verify whether a particular feature of the system works as intended. They help ensure that requirements are met, identify defects, and validate the overall quality of the software.

3.3.1. Test case 1: Dataset & File Validation

Table 3-1 Dataset & File Validation

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E1-US1-TC1	28000 clips + valid paths	0–28000 clips	All clips loaded and mapped to metadata	Pass
E1-US1-TC2	0 clips	0–28000 clips	Error: “No dataset found”	Pass
E1-US1-TC3	500 missing audio files	0–28000 missing	Missing rows flagged as invalid	Pass

Description: This range specifies the valid dataset size boundary (from empty to full SEP-28K), used to verify behavior for no data, partial data and full-dataset conditions.

3.3.2. Test Case 2: Broken URL Detection

Table 3-2 Broken URL Detection

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E1-US2-TC1	0 broken URLs	0–100% broken	All URLs marked OK	Pass

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E1-US2-TC2	20% URLs return 404	0–100% broken	20% rows flagged as broken and excluded	Pass
E1-US2-TC3	Null / empty URL field	0–100% broken	Rows with empty URL flagged as missing and removed	Pass

Description: This percent range indicates the proportion of broken or unreachable URLs to test how the pipeline handles increasing levels of missing or invalid audio.

3.3.3. Test case 3: Annotation Agreement Validation

Table 3-3 Annotation Agreement Validation

TC ID	Input (Agreement %)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E1-US3-TC1	100% agreement	0–100%	Clip labeled as consistent, kept as is	Pass
E1-US3-TC2	40% agreement	0–100%	Clip tagged for special handling	Pass
E1-US3-TC3	66% (2 out of 3)	0–100%	Majority label applied to clip	Pass

TC ID	Input (Agreement %)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E1-US3-TC4	0% agreement (all annotators selected different labels)	0–100%	System unable to determine a majority label; clip flagged as ambiguous and excluded from training	Fail

Description: ECP: 0–100% agreement range denotes annotator agreement percentage and is used to decide when labels are consistent, when majority voting applies, or when clips need special review.

3.3.4. Testcase 4: Duplicate EP ID Check

Table 3-4 Duplicate EP ID Check

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US1-TC1	EP=1	no duplication EP ID	Only labeled in SEP- 28k dataset	Pass
E2-US1-TC2	EP=7	no duplication EP ID	Only labeled in Fluency dataset	Pass
E2-US1-TC3	EP=2	0no duplication EP ID	Exception (“Found in both dataset labels”)	Pass

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US1-TC4	EP=2 found in both SEP-28k and Fluency labels	No duplication EP ID	Exception: “Found in both dataset labels”	Fail

Description: This test case checks whether the same episode ID exists in both SEP-28k and FluencyBank datasets. The system should label the episode only in its correct dataset and show an exception if duplication is found.

3.3.5. Test case 5: Sample Rate Check

Table 3-5 Sample Rate Check

TC ID	Input (SampleRate)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US2-TC1	16000Hz	0–16000 Hz	Already standard, no resampling	Pass
E2-US2-TC2	44100 Hz	0–16000 Hz	Resampled to 16000 Hz	Pass
E2-US2-TC3	0Hz	0–16000 Hz	Invalid audio, clip rejected	Pass
E2-US2-TC4	8000 Hz	0–16000 Hz	Audio resampled to 16000 Hz	Fail

Description: This test case verifies whether audio files are correctly checked and resampled to the required 16000 Hz format. The system should keep valid 16000 Hz audio unchanged, resample higher sample rates, and reject invalid audio files.

3.3.6. Test case 6: Clip Start and End Check

Table 3-6 Clip Start and End Check

TC ID	Input (Clip samples)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US3-TC1	Start=11271840, End=13705600	Start<End(sample)	Clip trimmed successfully	Pass
E2-US3-TC2	Start=1370560, End=11271840	Start<End(sample)	Bad start/stop bound	Fail
E2-US3-TC3	Start=5000, End=5000	Start<End(sample)	Bad start/stop bound	fail

Description: Start<End(samples)

Start sample must be shorter than End sample of clip.

3.3.7. Test case 7: Annotation Vote Validation

Table 3-7 Annotation Vote Validation

TC ID	Input (Class votes)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US4-TC1	NoStutter=3 (Max)	0–3 votes	clip saved in NoStutter class	Pass
E2-US4-TC2	Block=1 (Min)	0–3 votes	clip will not save in Block class	Pass

TC ID	Input (Class votes)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US4-TC3	Each class has 1 vote	0–3 votes	Exception (“This clip has equal votes in each class”)	Pass
E2-US4-TC4	Block = 4	0-3 votes	Error: Invalid Vote count	Fail

Description: This test case verifies how annotator votes are used to assign a clip to the correct class. The system should save the clip only when one class has the highest votes and show an exception when votes are equal across classes.

3.3.8. Test case 8: Class Balancing

Table 3-8 Class Balancing

TC ID	Input (Class Clips)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US5-TC1	2500	3000 clips	class will do oversampling.	Pass
E2-US5-TC2	5000	3000 clips	class will do undersampling.	Pass
E2-US5-TC3	0	0-3000 clips	Error: No clips available for balancing	Fail

Description: This test case verifies that each class is balanced to the required 3000 clips. If a

class has fewer clips, the system performs oversampling, and if it has more clips, the system performs undersampling.

3.3.9. Test case 9: Audio Error Handling

Table 3-9 Audio Error Handling

TC ID	Input (Error Condition)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US6-TC1	Missing ClipId column	Present / Missing	ClipId auto-generated from row index	Pass
E2-US6-TC2	Missing required column (EpId, Start, Stop)	Valid / Missing	Execution stops with error: Missing required column	Pass
E2-US6-TC3	Missing label column	Valid / Missing	Execution stops with error: Missing label column	Pass
E2-US6-TC4	All label votes = 0	0–N votes	Row dropped from dataset	Pass

E2-US6-TC5	Output clip already exists	Exists / Not exists	Clip skipped, counted as skipped exists	Pass
E2-US6-TC6	Episode <code>.wav</code> file missing	Exists / Missing	Clip skipped, counted as missing_ep	Pass
E2-US6-TC7	Start $\geq$ Stop	0–N samples	Clip discarded, logged as bad bounds	Pass
E2-US6-TC8	Start < 0	Negative / Valid	Start clamped to 0 safely	Pass
E2-US6-TC9	Corrupt or unreadable audio file	Valid / Corrupt	Exception caught, counted as read errors	Pass
E2-US6-TC10	Missing required column but processing continues	Valid / Missing	Execution stops with error: Missing required column	Fail

Description: This test case checks how the system handles different errors during dataset preprocessing. It verifies missing columns, missing labels, invalid start/stop values, duplicate clips, missing audio files, and corrupt audio. The system should automatically fix small issues like negative start time or stereo audio, while serious errors should stop execution or skip the invalid clip. This helps keep the final dataset clean, valid, and ready for model training.

3.3.10. Test case 10: Class Standardization

Table 3-10 Class Standardization

TC ID	Input (Install result)	ECP (Codes)	Expected Output	Actual Result (Pass/Fail)
E2-US7-TC1	Class contains >3200 samples	Samples >3200	System randomly selects samples to reduce class size to 3200	Pass
E2-US7-TC2	A speech class contains exactly 3200 samples	samples= 3200	Class remains unchanged with 3200 samples	Pass
E2-US7-TC3	A speech class contains fewer than 3200 samples	Samples <3200	System duplicates samples to reach 3200	Pass
E2-US7-TC4	Class contains 0 samples	0 samples	Error: No samples available for balancing	Fail

Description: This test case verifies that all speech classes are standardized to 3200 samples. Classes with excess samples are reduced through undersampling, while classes with insufficient samples are expanded through oversampling.

3.3.11. Test case 11: Audio Length Standardization

Table 3-11 Audio Length Standardization

TC ID	Input (Clip length)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
-------	------------------------	----------------	-----------------	------------------------------

E3-US1-TC1	1.5 seconds	0–3 seconds	Clip padded with zeros to exactly 3 seconds (48,000 samples)	Pass
E3-US1-TC2	4.2 seconds	>3 seconds	Clip truncated to exactly 3 seconds (48,000 samples)	Pass
E3-US1-TC3	Corrupted audio file	Null/Invalid	Exception caught, file skipped, returns None	Fail
E3-US1-TC4	Empty audios/ 0 sec	Null/Invalid	File rejected and returns None	Fail

Description: This test case verifies that every audio clip is converted to a fixed 3-second duration before feature extraction. Short clips are padded, long clips are truncated, and corrupted files are skipped safely.

3.3.12. Test case 12: Feature Matrix Generation

Table 3-12 Feature Matrix Generation

TC ID	Input (Feature Components)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E3-US2-TC1	WordRep audio clips, 48k samples at 16kHz	0.50–0.65 F1-score	Features extracted with acceptable WordRep detection	F1-score 0.580279, Recall 0.562; repetitions missed (fail)

E3-US2-TC13	NaturalPause audio clips,	0.40–0.60 F1-score	Natural pauses classified correctly	F1-score 0.489679, Recall 0.427; weak pause detection (fail)
E3-US2-TC15	Natural pause vs stutter block detection	recall < 0.50	System should distinguish pauses from blocks	Natural pauses confused with stutter blocks(fail)
E3-US2-TC1	Prolongation audio clips,	0.80–1.00 F1-score	Prolongation features extracted successfully	F1-score 0.811127, Recall 0.831 (PASS)
E3-US2-TC4	Interjection audio clips,	0.70–0.80 F1-score	Interjection/filler features extracted successfully	F1-score 0.724851, Recall 0.789 (PASS)
E3-US2-TC7	Block audio clips,	0.70–0.80 F1-score	Block features extracted successfully	F1-score 0.721457, Recall 0.733 (PASS)

Description: These test cases evaluate the 215-combo feature extraction pipeline across different stutter classes. Prolongation, Interjection, and Block achieved acceptable F1-score and recall values, so they passed the feature validation. WordRep and NaturalPause showed lower recall, meaning repeated words and natural pauses were not captured reliably.

3.3.13. Test case 13: Feature Saving and Management

Table 3-13 Feature Saving and Management

TC ID	Input (Class Directory)	ECP (Files)	Expected Output	Actual Result (Pass/Fail)
E3-US3-TC1	"Block" folder with 500 clips	>0 files	Aggregated array saved successfully as Block_zcr_v2_X.npy	Pass

E3-US3-TC2	"NaturalPause" folder with 0 clips	0 files	Warning logged: "No .wav files found", no .npy file saved	Pass
E3-US3-TC3	Memory after saving batch	Memory cleanup	Memory cleared successfully using gc.collect()	Pass

Description: (ECP: >0 files, 0 files, memory cleanup)

This test case verifies that extracted audio features are aggregated, saved as class-wise .npy files, and properly managed in memory during batch processing.

3.3.14. Test case 14: Dataset Scaling

Table 3-14 Dataset Scaling

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E4-US1-TC1	Raw .npy array	Min-Max Scaling	Values normalized safely to prevent NaN/Inf errors	Pass
E4-US1-TC2	Class with 3000 samples	Target: 5000	Oversampled with replacement to exactly 5000	Pass
E4-US1-TC3	Train/Test Split ratio	Test_size=0.20	Dataset split exactly 80% train / 20% test	Pass
E4-US1-TC4	Invalid train/test ratio (e.g., 90/10	Test_size=0.20	Dataset split exactly 80% train / 20% test	Fail

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
	instead of 80/20)			

Description: This test case verifies that feature arrays are normalized using Min-Max scaling, classes are balanced to the required sample count, and the final dataset is split into 80% training and 20% testing data.

3.3.15. Test case 15: Model Selection and Performance Evaluation

Table 3-15 Model Selection and Performance Evaluation

TC ID	Input	ECP (Regularization)	Expected Output	Actual Result(Pass/Fail)
E4-US2-TC1	BiLSTM model with attention	0.90–1.00	Model should meet final accuracy target	Accuracy ~89%; below target and slower for real-time use (FAIL)
E4-US2-TC2	Random Forest model	0.90–1.00	Model should meet final accuracy target	Test accuracy ~75%; overfitting detected (FAIL)
E4-US2-TC3	Logistic Regression model	0.90–1.00	Model should meet final accuracy target	Accuracy ~44%; critical underfitting (FAIL)

TC ID	Input	ECP (Regularization)	Expected Output	Actual Result(Pass/Fail)
E4-US2-TC4	CNN model with 215 features	0.90–1.00	Model should achieve high accuracy	Accuracy 94.43%; Macro-F1 0.9209 (PASS)
E4-US2-TC5	CNN training convergence	0.90–1.00	Validation accuracy should improve	Validation accuracy reached 0.948 (PASS)

Description: This test case compares different machine learning models against the required performance range. It validates that weaker models fail due to low accuracy, overfitting, or real-time limitations, while the CNN model meets the final accuracy and convergence targets.

3.3.16. Test case 16: Model Evaluation

Table 3-16 Model Evaluation

TC ID	Input (Training Event)	ECP (Condition)	Expected Output	Actual Result (Pass/Fail)
E4-US3-TC1	Epoch ends (val_acc improves)	save_best_only=True	Model weights saved to stutter_robust_final.k eras	Pass

TC ID	Input (Training Event)	ECP (Condition)	Expected Output	Actual Result (Pass/Fail)
E4-US3-TC2	Epoch ends (val_acc drops)	save_best_only=True	Previous best weights retained; no overwrite	Pass
E4-US3-TC3	Evaluation on X_test	y_pred vs y_test	Classification report generated and printed to console	Pass

Description; This test case verifies that the best model weights are saved only when validation accuracy improves. It also ensures that final evaluation is performed on test data and a classification report is generated.

3.3.17. Test case 17: Microphone Permission Handling

Table 3-17 Microphone Permission Handling

TC ID	Input (Mic Permission)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E5-US1-TC1	1	0–1	Continuous audio stream received	Pass
E5-US1-TC2	0	0–1	Show “Microphone access required”	Pass
E5-US1-TC3	Stream timeout	0–1	Monitoring restarted, error logged	Pass

Description: (ECP: MicPermission 0–1) This binary range represents microphone permission state and is used to test app behavior when permission is granted (1) or denied (0).

3.3.18. Testcase 18: Real-Time Stutter Detection

Table 3-18 Time Stutter Detection

TC ID	Input (Frame type)	ECP (Codes)	Expected Output	Actual Result (Pass/Fail)
E5-US2-TC1	Clear stutter pattern	0 = normal, 1 = Stutt.	Output = “stutter”, timestamp saved	Pass
E5-US2-TC2	Fluent speech	0 = normal, 1 = Stutt.	Output = “no stutter”	Pass
E5-US2-TC3	Very noisy frame	0 = normal, 1 = Stutt.	Marked as “uncertain / ignored”	Pass
E5-US2-TC4	Clear stutter pattern classified as normal speech	0 = normal, 1 = Stutter	Output = “stutter”, timestamp saved	Fail

Description: (ECP: codes 0 = normal, 1 = stutter) This categorical code range denotes classification labels used to verify that frames are labeled correctly as “normal” or “stutter” (and to handle uncertain frames).

3.3.19. Testcase 19: Prediction Trigger and Control

Table 3-19 Prediction Trigger and Control

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E6-US1-TC1	Stutter detected as WordRep	Stutter Flag = 1	Prediction module activates automatically	Pass
E6-US1-TC2	Stutter detected as Prolongation	Stutter Flag = 1	Prediction module activates automatically	Pass
E6-US1-TC3	Stutter detected as Block	Stutter Flag = 1	Prediction module activates automatically	Pass
E6-US1-TC4	Fluent speech detected	Stutter Flag = 0	Prediction module does not activate	Pass
E6-US1-TC5	Multiple stutters detected rapidly	Active Prediction = 1	Duplicate prediction requests are ignored	Pass
E6-US1-TC6	Prediction module is active	Listening State = Paused	Live transcription temporarily pauses until selection is completed	Pass

Description: This test case verifies that the prediction module activates only when a stutter event is detected. It also ensures duplicate prediction requests are ignored and live transcription pauses while prediction selection is active.

3.3.20. Testcase 20: Word Prediction and Suggestion Handling

Table 3-20 Word Prediction and Suggestion Handling

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E6-US2-TC1	Valid prefix: “I want to”	Prefix = Valid	Ranked word suggestions are generated	Pass
E6-US2-TC2	Empty prefix after isolated sound stutter, e.g., “sss”	Prefix = Empty	Default/fallback suggestions are generated	Pass
E6-US2-TC3	Backend returns fewer than 3 suggestions	Suggestions = 1–2	Available suggestions are displayed without crashing	Pass
E6-US2-TC4	Backend returns no suggestions	Suggestions = 0	Fallback suggestions are displayed	Pass
E6-US2-TC5	User selects one suggestion	Selection = 1	Selected word is inserted into transcript	Pass
E6-US2-TC6	API request fails	API Status = Failed	Fallback suggestions are displayed and app remains stable	Pass

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E6-US2-TC7	Prediction selection completes	Listening State = Resume	Continuous speech listening resumes automatically	Pass
E6-US2-TC8	User selects one suggestion but transcript is not updated	Selection = 1	Selected word is inserted into transcript	Fail

Description: This test case verifies that the system generates ranked word suggestions from a valid or empty prefix, handles missing/API-failed suggestions using fallback words, inserts the selected suggestion into the transcript, and resumes listening after selection.

3.3.21. Testcase 21: Live Speech Session Control

Table 3-21 Live Speech Session Control

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E7-US1-TC1	Tap mic button with permission granted	Permission = 1	Live speech monitoring starts immediately	Pass
E7-US1-TC2	User speaks continuously	Transcript State = Active	Transcript updates continuously in real time	Pass

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E7-US1-TC3	Tap stop button during active session	Recording State = Stop	Monitoring and recording stop safely	Pass
E7-US1-TC4	Microphone permission denied	Permission = 0	Warning message displayed	Pass

Description: (ECP: 0–1 active/ stop timing) This testcase verifies that the user can start and stop live speech monitoring using the mic controls. It also checks real-time transcript updates and proper handling of denied microphone permission.

3.3.22. Testcase 22: Prediction Display

Table 3-22 Prediction Display

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E7-US2-TC1	WordRep detected	Detection State = Active	Detected stutter type displayed instantly	Pass
E7-US2-TC2	Backend returns 3 suggestions	Suggestions = 3	Suggestions displayed as tappable chips	Pass
E7-US2-TC3	User selects prediction chip	Selection = 1	Selected word inserted into transcript	Pass

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E7-US2-TC4	Prediction completes	Listening State = Resume	Status changes back to “Listening...”	Pass
E7-US2-TC5	Backend returns 3 suggestions but chips are not displayed	Suggestions = 3	Suggestions displayed as tappable chips	Fail

Description: (ECP: Suggestions 0–3 / Session States). This test case verifies that detected stutter types and prediction suggestions are displayed instantly in the UI. It also checks that selected suggestions are inserted into the transcript and the interface remains responsive during continuous prediction cycles.

3.3.23. Testcase 23: Doctor Dashboard Event Display

Table 3-23 Doctor Dashboard Event Display

TC ID	Input (Stutter events)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E8-US1-TC1	5 events in session	0–N	5 events displayed in doctor feed in real time	Pass
E8-US1-TC2	0 events	0–N	Feed stays empty	Pass
E8-US1-TC3	Network delay	0–N	Events appear with slight delay but not lost	Pass

Description: (ECP: 0–N events) This range represents the number of stutter events in a session and is used to check whether the doctor feed correctly displays any number of events including none.

3.3.24. Testcase 24: Doctor-Patient Prediction Sync

Table 3-24 Doctor-Patient Prediction Sync

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E8-US2-TC1	Patient suggestions [3]	0–3	Same 3 suggestions visible on doctor screen	Pass
E8-US2-TC2	Patient selects 1 word	0–3	Selected word highlighted on doctor interface	Pass
E8-US2-TC3	Sync error	0–3	Show sync error; system retries fetching data	Pass
E8-US2-TC4	Patient suggestions [3] not visible on doctor screen	0–3	Same 3 suggestions visible on doctor screen	Fail

Description: (ECP: 0–3 suggestions) This range again represents the allowable number of suggestions visible to the doctor, used to confirm synchronization of suggestion lists.

3.3.25. Testcase 25: Live Monitor Synchronization

Description: This test case verifies whether the live session data is synced correctly between patient and monitor interfaces. The system should instantly update transcript, stutter status, and recording state on both devices.

Table 3-25 Live Monitor Synchronization

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
EP9-US1- TC1	Live transcript update	Session Active = True	Transcript instantly synced to monitor interface	Pass
EP9-US1- TC2	Stutter detected as WordRep	Detection State = Active	Same stutter status displayed on both devices	Pass
EP9-US1- TC3	Recording started	Recording State = True	Both interfaces show active recording state	Pass
EP9-US1- TC4	Recording stopped manually	Recording State = False	Both interfaces show recording paused	Pass

Description: This test case verifies that generated prediction suggestions are synchronized between patient and monitor interfaces. It also ensures selected words, empty prediction lists, and rapid Firebase updates are handled smoothly.

3.3.26. Testcase 26: Firebase Prediction Update Validation

Description: These ranges validate synchronization of prediction suggestions, selected words, and Firebase recovery after reconnection.

Table 3-26 Firebase Prediction Update Validation

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
EP9-US2-TC1	Context-related prediction suggestions generated	Prediction Count = 0–5	Same prediction chips visible on monitor screen	Pass
EP9-US2-TC2	Patient selects one prediction word	Selected Prediction = 1	Selected word instantly synced to both interfaces	Pass
EP9-US2-TC3	Empty prediction array	Prediction Count = 0	No prediction chips displayed without crashing	Pass
EP9-US2-TC4	Multiple rapid Firebase updates	Sync Events = Rapid	Synchronization remains stable and responsive	Pass

3.3.27. Testcase 27: Selected Prediction Sync Validation

Description: This test case verifies whether Firebase is correctly connected with the Flutter Android application. The system should create the project, generate configuration files, install packages, and enable Realtime Database successfully.

Table 3-27 Selected Prediction Sync Validation

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
EP9-US3-TC1	Firebase project created	Project Status = Created	Firebase project available in Firebase Console	Pass
EP9-US3-TC2	FlutterFire CLI configured	Configuration = Success	firebase_options.dart generated successfully	Pass
EP9-US3-TC3	Android app registered	Android Config = Added	google-services.json added to Android project	Pass
EP9-US3-TC4	Firebase packages installed	Dependency Status = Installed	Firebase packages installed without errors	Pass
EP9-US3-TC5	Realtime Database enabled	Database Status = Enabled	App can write session data to Firebase	Pass

3.3.28. Testcase 28: Monitor Interface Stream

Description: This test case verifies whether the monitor interface correctly receives and displays live Firebase updates. The system should update transcript, predictions, and recording status in real time without crashes or delays.

Table 3-28 Monitor Interface Stream

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
EP9-US4-TC1	Monitor screen opened with active session	Firebase Data = Available	Latest transcript displayed automatically	Pass
EP9-US4-TC2	Firebase prediction update received	Stream Event = Prediction Update	Prediction chips update on monitor screen	Pass
EP9-US4-TC3	Firebase recording state update received	Recording State = True/False	Monitor shows correct live or paused state	Pass
EP9-US4-TC4	Firebase has no session data	Firebase Data = Null	Waiting state displayed without crashing	Pass
EP9-US4-TC5	Continuous Firebase stream events	Stream Events = Multiple	Monitor UI refreshes smoothly	Pass

3.3.29. Testcase 29: Functional Testing

Table 3-29 Functional Testing

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E10-US1-TC1	All module test cases executed	0–100% pass rate	100% pass rate → Testing PASSED	100% test cases passed successfully
E10-US1-TC2	Stutter audio from all classes	0–100% accuracy	Correct stutter class detected ($\geq 94.3\%$ accuracy)	Model achieved 94.3% accuracy
E10-US1-TC3	Active Firebase session	7 fields	All required session fields updated correctly	All 7 fields synchronized correctly
E10-US1-TC4	70% tests pass, 30% fail	0–100% pass rate	Failed modules identified and re-tested	Failed modules fixed and re-tested successfully
E10-US1-TC5	No test cases executed	0–100% pass rate	Error shown; testing marked incomplete	QA warning generated and testing halted

Description: (ECP: 0–100% pass rate) This range defines the overall functional test pass rate. A 100% pass rate confirms all modules are working as specified; lower values indicate defects requiring resolution before deployment approval.

3.3.30. Testcase 30: Real-Time Performance Testing

Table 3-30 Real-Time Performance Testing

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E10-US2-TC1	150 ms	0–1000 ms	Real-time OK	Result returned in 150 ms (Pass)
E10-US2-TC2	300 ms	0–1000 ms	Suggestions displayed within limit	Suggestions shown in 300 ms (Pass)
E10-US2-TC3	Firestore sync latency	0–1000 ms	Sync updated without delay	Transcript and predictions synced successfully (Pass)
E10-US2-TC4	1200 ms	0–1000 ms	Performance FAIL	Latency exceeded limit (Fail)
E10-US2-TC5	No response	0–1000 ms	Pipeline failure flagged	No response received from engine (Fail)

Description: (ECP: 0–1000 ms latency) This range defines the acceptable real-time response window for stutter detection, word prediction, and Firestore synchronization. Values within the lower bound confirm real-time performance; values above the upper bound indicate failures requiring optimization.

3.3.31. Testcase 31: Deployment Validation

Table 3-31 Deployment Validation

TC ID	Input	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E10-US3-TC1	Install APK on Samsung SM-A325F	Install Status: 0/1	App installed successfully	Installation successful (Pass)
E10-US3-TC2	Deploy APK on Pixel 9 Pro XL Emulator	Install Status: 0/1	App runs correctly on emulator	Installation and execution successful (Pass)
E10-US3-TC3	APK rejected by device	Install Status: 0/1	Installation failed	APK installation failed (Fail)
E10-US3-TC4	3 crashes during testing	Crash Count: 0–N	Deployment rejected	Crashes detected; deployment failed (Fail)
E10-US3-TC1	Install APK on Samsung SM-A325F	Install Status: 0/1	App installed successfully	Installation successful (Pass)

Description: (ECP: install/crash codes 0/1 and 0–N crashes) These codes define deployment readiness. A successful install (status 1) with zero crashes across both the Samsung SM-A325F physical device and the Pixel 9 Pro XL emulator confirms that the application is stable and approved for release.

3.4. User Interface Design

3.4.1. UI- Home Screen

This welcome screen is the entry point of the Speech to Text Predictor app, where the user first lands. It shows a clear title, explains that the app will transform voice into intelligent text suggestions, and asks for microphone permission with a dedicated button.



Figure 3.4.1. Home Screen

3.4.2. UI-2. Session Selection

This screen allows the user to start or join a real-time speech **session** with a listener. They can instantly generate a unique session code or enter an existing one to connect.

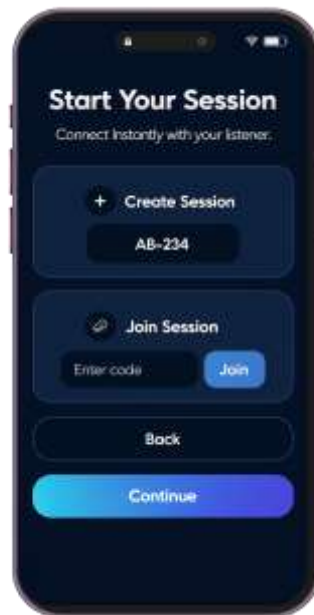


Figure 3.4.2 Session Selection

3.4.3. UI-3. Patient Section

This screen lets the patient speak naturally while the system converts their voice into text in real time.



Figure 3.4.3. Patient Section

3.4.4. UI-4. Doctor Section

The Monitor screen enables a doctor, therapist, or observer to view a speaker's speech activity in real time. It displays the live transcription received from the speaker's device, allowing the monitor to track communication progress.

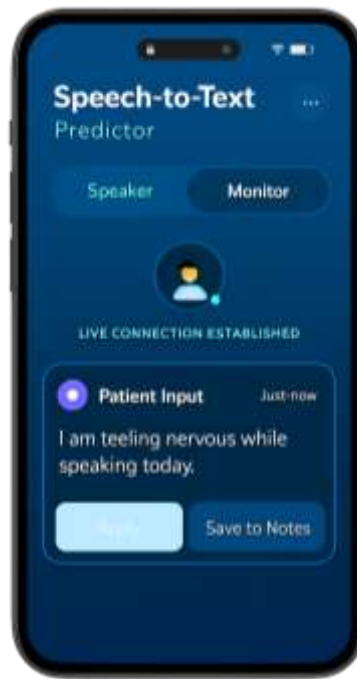


Figure 3.4.4, Doctor Section

3.5. Traceability Matrix

Traceability Matrix is a structured table that establishes the relationship between different artifacts of the system, including Epics, User Stories, Test Cases, and User Interfaces (UIs). It ensures that each epic is broken down into user stories, validated through test cases, and supported by appropriate UI components. This matrix helps maintain consistency, verifies coverage of requirements, and ensures that no functionality is missed during development.

Table 3-5 Traceability Matrix

Epic	User Story	Test Case ID	User Interface
------	------------	--------------	----------------

E1 – Dataset Acquisition	E1-US1: Dataset Collection	E1-US1-TC1 to E1-US1-TC3	
	E1-US2: Broken & Missing Audio Detection	E1-US2-TC1 to E1-US2-TC3	
	E1-US3: Collecting Valid Audios	E1-US3-TC1 to E1-US3-TC4	
E2 – Audio Preprocessing	E2-US1: Label Cleaning	E2-US1-TC1 to E2-US1-TC4	
	E2-US2: Audio Standardization	E2-US2-TC1 to E2-US2-TC4	
	E2-US3: Trimming Clips from Episode Audios	E2-US3-TC1 to E2-US3-TC3	
	E2-US4: Adding Clips to Maximum Class	E2-US4-TC1 to E2-US4-TC4	
	E2-US5: Merge and Balancing Class	E2-US5-TC1 to E2-US5-TC3	
	E2-US6: Robust Exception Handling	E2-US6-TC1 to E2-US6-TC13	
	E2-US7: Class Balancing of Processed Dataset	E2-US7-TC1 to E2-US7-TC4	

E3 – Feature Extraction	E3-US1: Fixed Length Audio Standardization	E3-US1-TC1 to E3-US1-TC4	
	E3-US2: Multi-Feature Extraction	E3-US2-TC1 to E3-US2-TC4	
	E3-US3: Class-wise Feature Aggregation and Storage	E3-US3-TC1 to E3-US3-TC4	
E4 – ML Model Training	E4-US1: Data Loading & Scaling	E4-US1-TC1 to E4-US1-TC4	
	E4-US2: Build CNN Model with Overfitting Prevention	E4-US2-TC1 to E4-US2-TC5	
	E4-US3: Hyperparameter Tuning	E4-US3-TC1 to E4-US3-TC3	
E5 – Stutter Detection Engine	E5-US1: Continuous Audio Monitoring	E5-US1-TC1 to E5-US1-TC3	
	E5-US2: Stutter Identification	E5-US2-TC1 to E5-US2-TC4	

E6 – Word Prediction Engine	E6-US1: Real-Time Prediction Trigger	E6-US1-TC1 to E6-US1-TC6	
	E6-US2: Context-aware Word Suggestion	E6-US2-TC1 to E6-US2-TC8	
E7 – Patient Mobile Application	E7-US1: Real-Time Speech Interaction	E7-US1-TC1 to E7-US1-TC5	Patient UI
	E7-US2: Live Prediction and Detection Display	E7-US2-TC1 to E7-US2-TC6	Patient UI
E8 – Doctor Monitoring Interface	E8-US1: Real-Time Stutter Feed	E8-US1-TC1 to E8-US1-TC4	Monitor UI
	E8-US2: Prediction Visibility	E8-US2-TC1 to E8-US2-TC4	Monitor UI
E9 – Real Time Synchronization	E9-US1: Real-Time Transcript & Stutter Synchronization	E9-US1-TC1 to E9-US1-TC5	Patient UI / Monitor UI
	E9-US2: Prediction & Selection Synchronization	E9-US2-TC1 to E9-US2-TC5	Patient UI / Monitor UI

	E9-US3: Firebase Project and App Configuration	E9-US3-TC1 to E9-US3-TC5	
	E9-US4: Live Monitor Listener Integration	E9-US4-TC1 to E9-US4-TC6	Monitor UI
E10 – Deployment and System Testing	E10-US1: Functional Testing	E10-US1-TC1 to E10-US1-TC2	
	E10-US2: Real-Time Performance Testing	E10-US2-TC1 to E10-US2-TC4	
	E10-US3: Deployment Validation	E10-US3-TC1 to E10-US3-TC4	

Chapter 4

4. Proposed Solution

As discussed in the previous chapters, existing stuttering support systems mainly focus on offline detection, long-term therapy, or hardware-based solutions, and they lack real-time assistance, mobile deployment, and predictive communication features. To address these limitations, this project proposes a real-time Speech-to-Text Predictor mobile application that detects stuttering in speech and predicts the intended words to support fluent communication. The system converts speech into text, identifies disfluency patterns, and instantly suggests possible words to the user. Using Firebase synchronization, the predicted output is shared in real time between the patient and doctor applications. By integrating speech processing, machine learning, and cloud technology, the proposed solution provides a practical and intelligent tool to assist individuals who stutter during live communication.

4.1. Software Architecture

The proposed Speech-to-Text Predictor system is designed using a layered architecture integrated with artificial intelligence and cloud-based synchronization services to support real-time speech analysis and communication assistance. The system consists of four major layers: the Application Layer, AI Processing Layer, Cloud & Database Layer, and Presentation Layer. The Application Layer handles microphone input, audio preprocessing, and feature engineering using 162-dimensional acoustic features. The AI Processing Layer contains the CNN-GRU-based stutter detection model, context analysis module, and context-related word prediction engine. The Cloud & Database Layer uses Firebase Realtime Database to synchronize transcripts, stutter status, and prediction suggestions in real time between connected devices. The Presentation Layer includes the Patient Mobile Application and Doctor/Monitor Interface, allowing both users to view synchronized live speech updates.

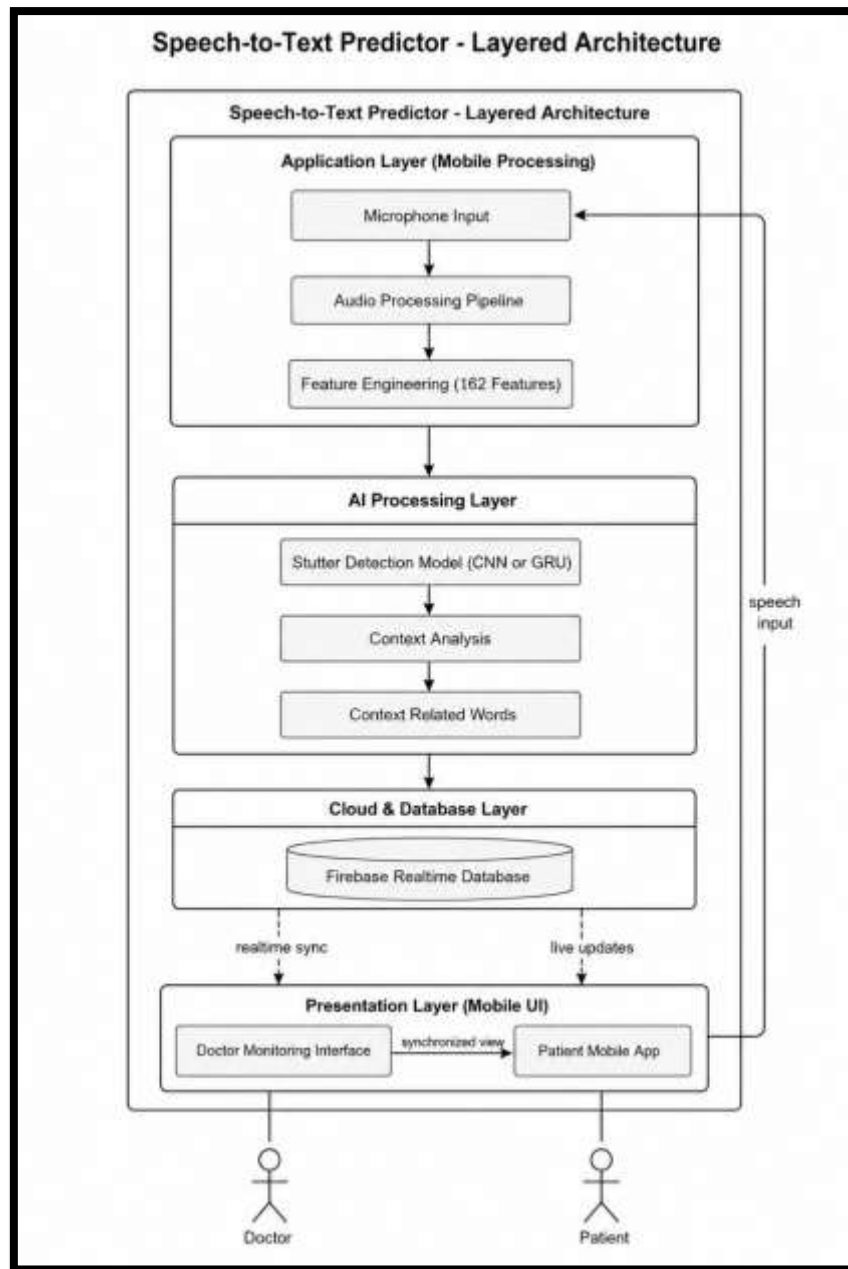


Figure 4.1 Software Architecture Diagram

4.2. Data Modeling

The Speech-to-Text Predictor system is designed around a session-based data model to support real-time speech processing and synchronization between patient and doctor applications. The

core entities in the system are Patient, Doctor, Session, and Prediction Event. Each patient can initiate multiple speech sessions, and each session may generate multiple prediction events during speech analysis. Session entities store information such as transcripts, suggested words, confidence scores, and timestamps, while Prediction Event entities record detailed stutter detection and word prediction results. This data model enables efficient storage, real-time updates, and structured representation of speech-related data.

4.2.1. Relational Databases

From a conceptual perspective, the system entities can be represented using a relational model. In this view, Patient and Doctor entities are associated with Session entities, and each Session is linked to multiple PredictionEvent records. This representation helps in understanding logical relationships and dependencies among entities, even though the system is not implemented using a traditional relational database.

Entity Relationship Diagram

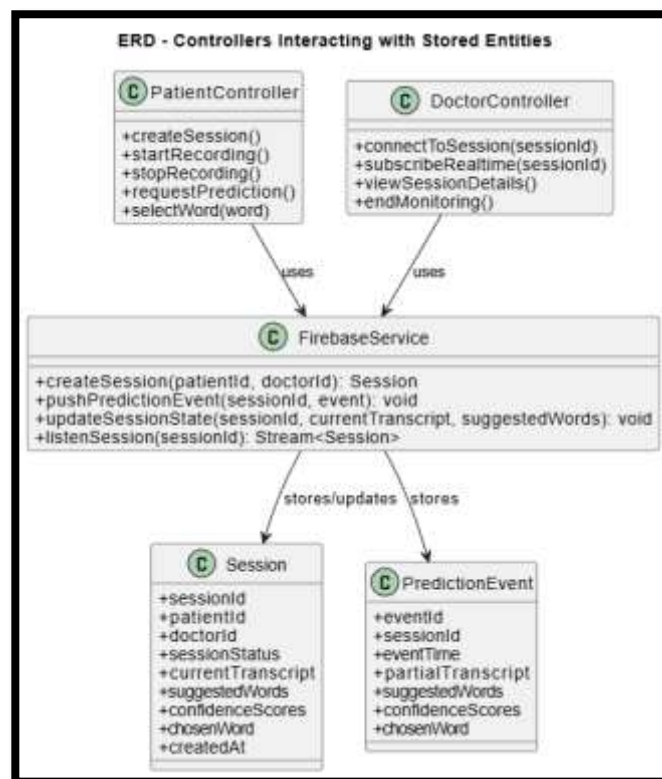


Figure 4-2 ERD

Database Schema

The following database schema diagram represents the hierarchical NoSQL structure of the Speech-to-Text Predictor system implemented using Firebase Realtime Database. The schema illustrates how user information, speech sessions, transcripts, stutter events, prediction suggestions, and real-time synchronization data are organized within the system. The proposed structure supports flexible data storage, scalability, and low-latency synchronization between patient and monitor applications during live communication sessions.

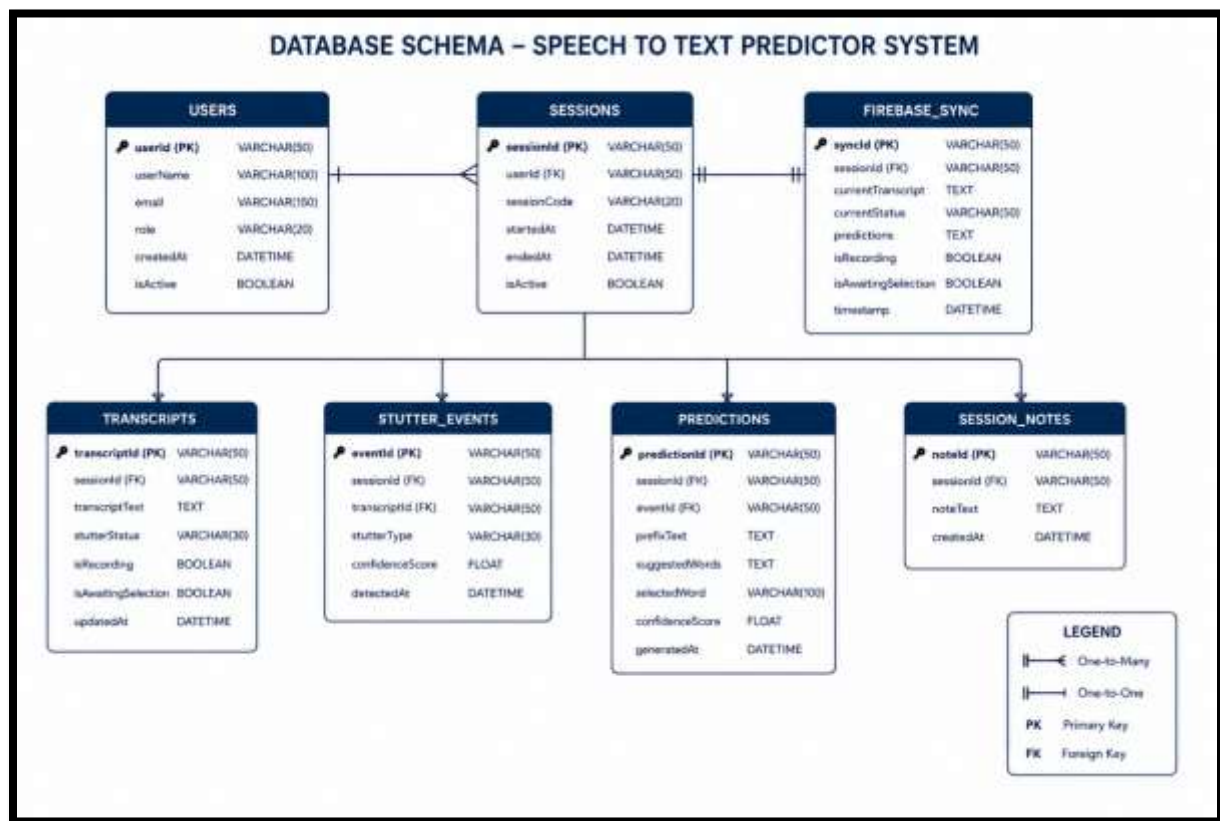


Figure 4.2.2 Database Schema Diagram

4.2.2. NoSQL Databases

The Speech to Text Predictor system is implemented using a NoSQL database model through

Firebase Realtime Database. Unlike relational databases, NoSQL databases store data in flexible, hierarchical structures such as JSON documents. This approach is suitable for real-time systems where data is dynamic and schema flexibility is required.

In the proposed system, Firebase stores information related to users, speech sessions, transcripts, stutter events, and predicted words. Each speech session is represented as a JSON node containing nested prediction events and transcript data. This design enables real-time synchronization between patient and doctor applications and supports scalable data management without complex table relationships.

Example (JSON Node Structure for Speech to Text Predictor)

```
{
  "users": {
    "patient_001": {
      "name": "Ali Khan",
      "role": "patient",
      "email": "ali.khan@example.com"
    },
    "monitor_001": {
      "name": "Dr. Ahmed",
      "role": "monitor",
      "email": "dr.ahmed@example.com"
    }
  },
  "sessions": {
    "session_001": {
      "patientId": "patient_001",
```

```
"monitorId": "monitor_001",

"transcript": "I want to call my friend",

"status": "WordRep",

"predictions": [

    "call",

    "contact",

    "talk" ],

"isRecording": true,

"isAwaitingSelection": true,

"timestamp": "2026-01-20T10:30:00Z",

"events": {

    "event_01": {

        "partialTranscript": "ca-ca",

        "detectedStutter": "WordRep",

        "predictions": [

            "call",

            "can",

            "come"

        ],

        "selectedWord": "call",

        "timestamp": "2026-01-20T10:30:10Z"

    }

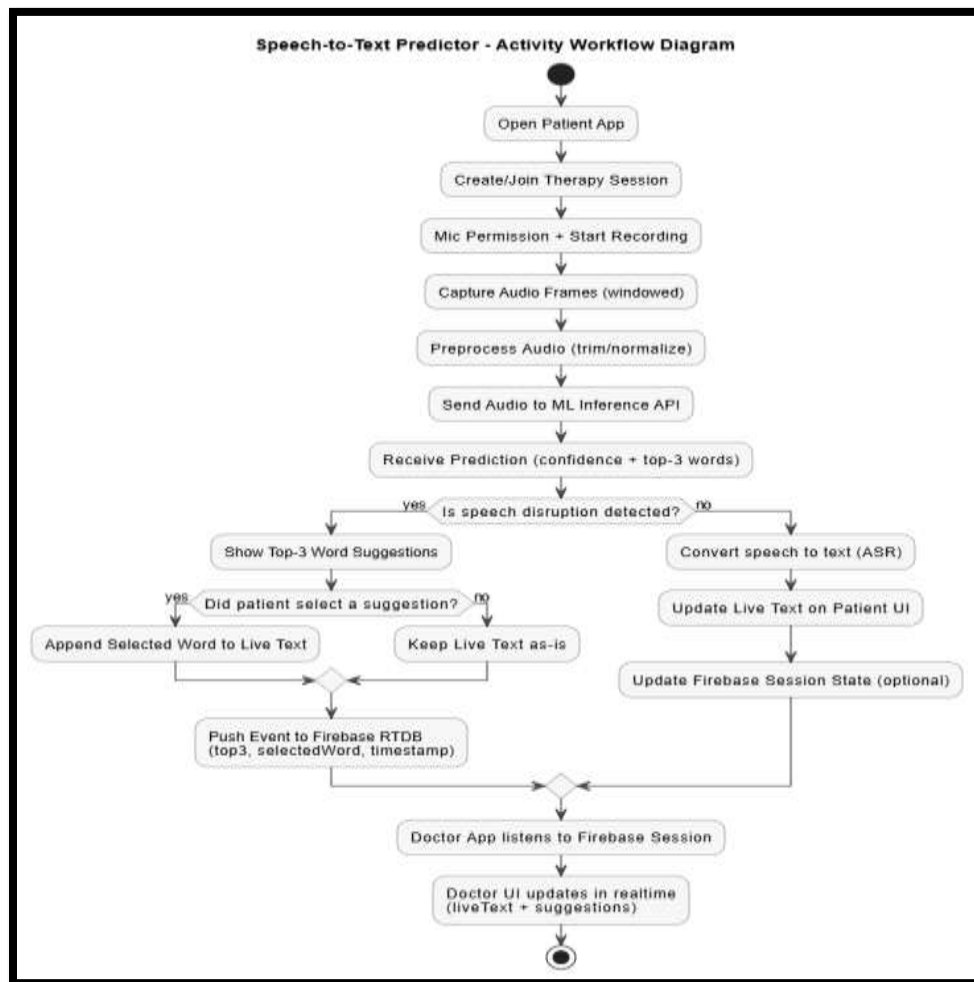
} }
```

This JSON structure demonstrates how speech sessions and prediction events are stored within Firebase. The hierarchical format allows efficient real-time updates and seamless data sharing between patient and doctor applications.

4.3. Workflow Diagram

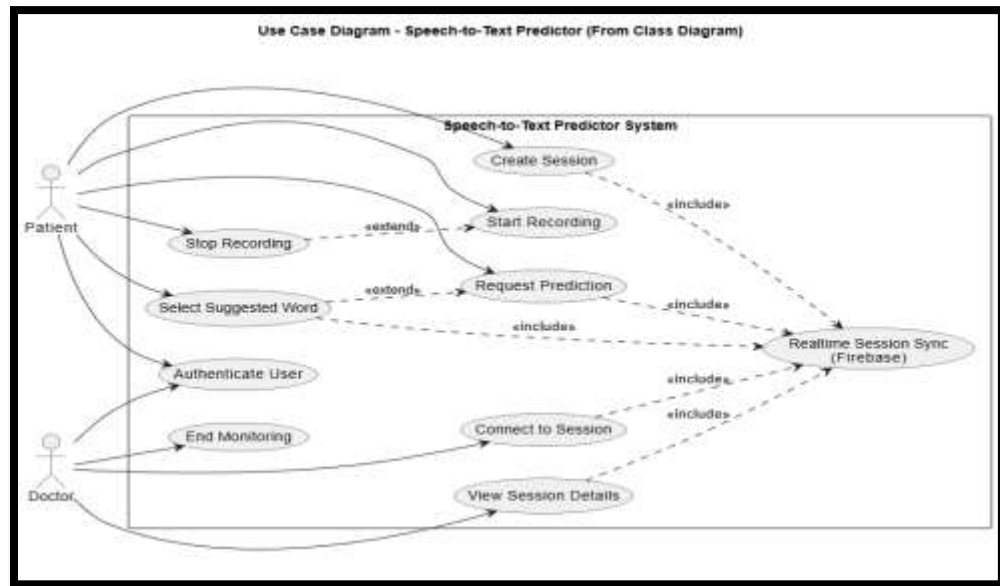
Our system is designed to support real-time speech analysis and communication assistance. The system processes speech input through multiple stages, including audio recording, preprocessing, stutter detection, word prediction, and data synchronization. The predicted results are displayed to the patient and simultaneously shared with the doctor application using Firebase, enabling real-time monitoring and interaction.

Figure 4.3 Activity diagram



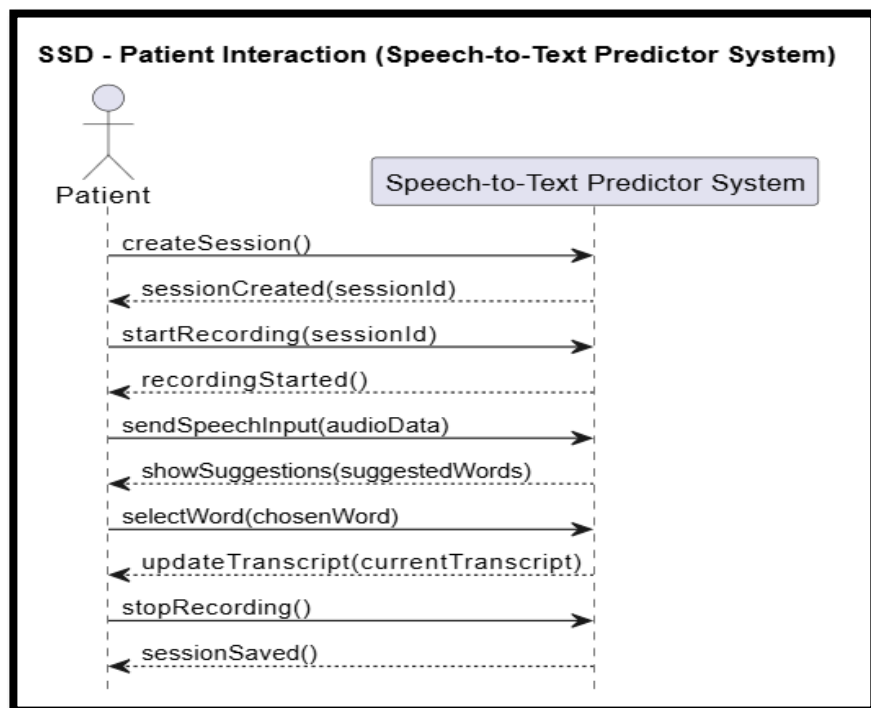
4.3.1. Use Case Diagram

Figure 4.1 Use case



4.3.2. Patient Interaction SSD

Figure 4.2 Patient SSD



4.3.3. Doctor Interaction SSD

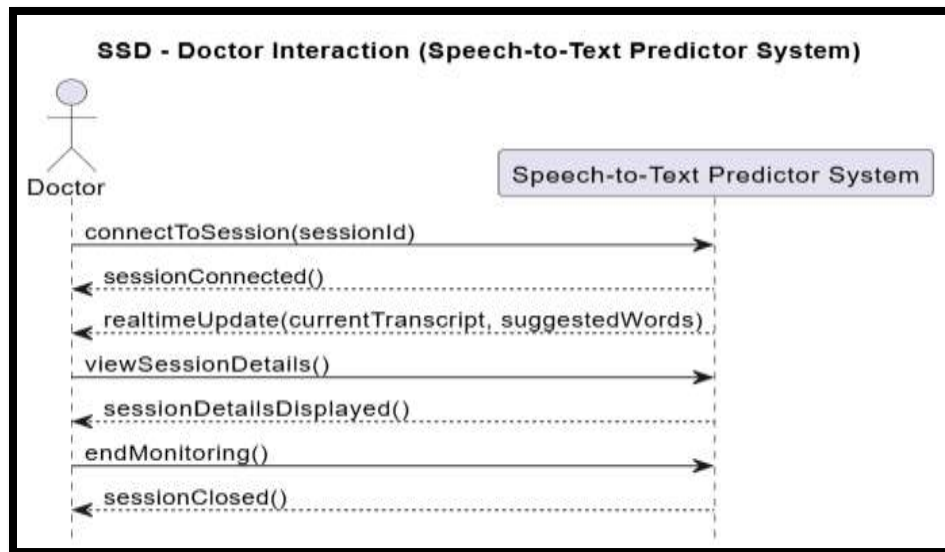


Figure 4.3 Doctor SSD

4.3.4. SequenceDiagram

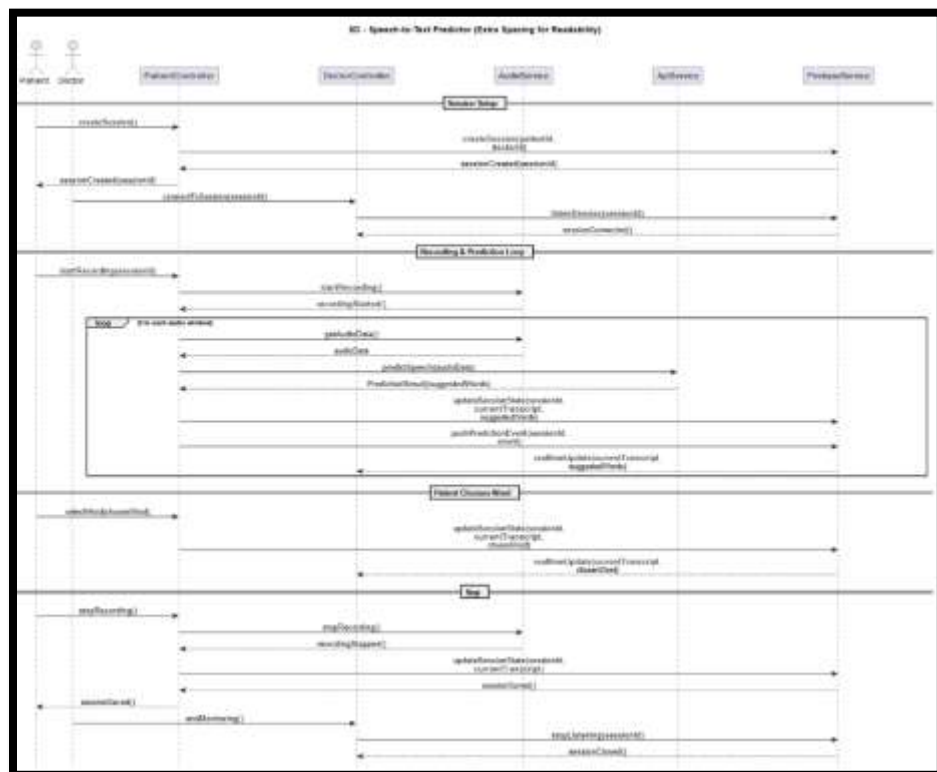


Figure 4.1 Sequence Diagram

4.4. Third-Parties Dependencies

The Speech-to-Text Predictor system relies on several external libraries, frameworks, and cloud services to support speech processing, mobile application development, and real-time data synchronization. The main third-party dependencies used in the system are:

1. **Firebase Authentication:** for secure user authentication and role management.
2. **Firebase Realtime Database:** for real time storage and synchronization of speech sessions and predictions.
3. **Librosa:** for audio signal processing and feature extraction.
4. **NumPy:** for numerical computation in speech processing.
5. **Scikit-learn:** for training and evaluation of machine learning models.
6. **Flutter SDK** – for developing the Android mobile application interface
7. **Speech-to-Text Package (speech_to_text):** used for converting live speech into text within the mobile application.
8. **TensorFlow Lite (tflite_flutter):** used for integrating lightweight machine learning models into the mobile application for on-device inference.
9. **Pandas:** used for dataset handling, preprocessing, and CSV manipulation.
10. **PyTorch:** used for deep learning model development, including CNN-GRU architectures.
11. **HTTP Package:** used for API communication between the Flutter application and backend prediction services.
12. **Record Package:** used for microphone audio recording and real-time audio capture within the mobile application.

13. **Permission Handler:** used for handling runtime permissions such as microphone access within the mobile application.

Chapter 5

5. Software Development

Our project, Speech-to-Text Predictor (STTP) system has progressed from the initial design phase into an advanced implementation and integration stage. The project now includes deep learning model development, real-time speech processing, Firebase cloud synchronization, and mobile application integration using Flutter. The implementation process focused on building a complete assistive communication system capable of detecting stuttering events and generating contextual word predictions during live speech sessions. During development, several technical challenges were encountered, particularly in speech preprocessing, feature engineering, dataset balancing, model optimization, and real-time synchronization. The raw audio dataset contained noise, silence, inconsistent recording quality, and uneven class distributions across different stutter categories. To address these issues, a preprocessing pipeline was designed that included audio cleaning, segmentation, normalization, and fixed-length standardization at 16 kHz. Class balancing techniques were also applied to reduce bias between stutter classes and improve model stability. Multiple acoustic feature extraction approaches were explored during experimentation. The final feature engineering pipeline combined Mel Spectrograms, MFCC coefficients, Chroma features, Spectral Centroid, and Zero Crossing Rate (ZCR), resulting in a 162-feature representation for each speech sample. This combination improved the system's ability to distinguish between fluent speech and multiple stutter patterns such as Blocks, Word Repetitions, and Prolongations. Several deep learning architectures were evaluated during development, including BiLSTM with Attention and CNN-GRU-based models for stutter detection and contextual prediction tasks. Hyperparameter tuning was performed to improve model performance by adjusting learning rates, batch sizes, dropout rates, optimizer configurations, focal loss parameters, and training epochs. Additional optimization techniques such as early stopping, class balancing, and learning-rate scheduling were also applied during experimentation.

The final optimized CNN-GRU deep learning model achieved approximately 98% accuracy during experimental evaluations after feature engineering, dataset balancing, and

hyperparameter optimization. Improved performance was achieved through the integration of combined acoustic features, focal loss optimization, and attention-based sequence learning techniques. The trained system demonstrated strong capability in recognizing multiple stutter patterns, including Blocks, Word Repetitions, Prolongations, and Interjections, while also generating contextual word prediction suggestions in real time. The mobile application component has also been integrated with Firebase Realtime Database to support real-time synchronization between patient and monitor interfaces. Live transcripts, prediction suggestions, recording states, and detected stutter events are now synchronized instantly between connected devices. The Flutter-based interface includes speech recording, transcript visualization, prediction selection, and monitor-side live session viewing.

At the current stage, the core functionalities of the proposed system have been successfully implemented and integrated. Remaining work primarily focuses on improving real-world robustness, optimizing inference speed for mobile deployment, and conducting extended testing across larger conversational environments.

5.1. Coding Standards

The following coding standards were followed during the development of the Speech-to-Text Predictor system:

i. Indentation

- Python: 4space indentation
- Flutter/Dart: standard Flutter indentation rules

ii. Naming Conventions

- Python variables & functions: snake_case(eg. patient_name)
- Flutter/Dart variables & methods: camelCase
- Class names: PascalCase
- File names: descriptive (e.g., feature_extraction.py, firebase_service.dart)

iii. Declarations & Structure

- Logical grouping of constants, variables, and functions
- Modular design with separate files and classes for each functionality

iv. Code Readability

- Meaningful variable and function names
- Comments added for complex logic (especially ML and Firebase code)

v. Error Handling

- Exception handling in Python
- Try catch blocks in Flutter/Dart

vi. Real-Time Processing

- Asynchronous programming (async/await) is used in Flutter to ensure smooth real-time performance.
- Timers and debounce mechanisms are used to manage continuous speech input and avoid excessive processing
- State management is carefully handled to prevent UI lag during live speech detection and prediction

vii. API Integration

- RESTful APIs are used for communication between the Flutter application and the Python backend
- JSON format is used for structured request and response handling
 - A separate service layer (e.g., `detection_service.dart`) is implemented to manage API calls
- Network errors and API failures are handled gracefully using exception handling

viii. Modular Hybrid Architecture

- Speech recognition, stutter detection, and prediction logic are implemented as independent modules
- This modular design allows easy updates or replacement of components (e.g., improving ML model without affecting UI)

These standards ensured that the code remained clean, maintainable, and easy to debug throughout the project.

5.2. Development Environment

Operating System:

Windows is used as the primary operating system because it provides stable support for python based machine learning tasks, Flutter development, and Firebase integration.

IDEs & Editors:

- **VS Code:** Used for speech data preprocessing, feature extraction, and machine learning model development due to its flexibility and strong Python support.
- **Android Studio:** Used exclusively for developing the Flutter-based Android mobile application because of its native Flutter and emulator support.
- **Google Collab:** Used for training and experimenting with machine learning models, especially for handling large datasets and computationally intensive tasks.

Programming Environments:

- Python environment is used for preprocessing, feature extraction, and model training.
- Flutter SDK is used for mobile application development.
- Firebase console for database and synchronization

Package Management:

- **pip**: Used to manage Python libraries such as librosa, numpy, pandas, scikit-learn, PyTorch, TensorFlow, and other machine learning dependencies required for speech preprocessing, feature extraction, and deep learning model training.
- **pub.dev**: Used as the official package repository for Flutter and Dart dependencies, providing packages such as `firebase_core`, `firebase_database`, `speech_to_text`, `tflite_flutter`, `permission_handler`, and `record` that were integrated into the mobile application for authentication, real-time synchronization, speech recognition, and audio recording functionalities.

Cloud Services:

Firestore Authentication and Firestore Realtime Database are used for secure user management and real-time synchronization between patient and monitor applications. Firestore Realtime Database is used to synchronize live transcripts, prediction suggestions, recording states, and detected stutter events between connected users in real time.

Development Setup:

The development environment was configured using Python in VS Code for machine learning development and Flutter in Android Studio for mobile application development. Firestore services were integrated with the Flutter application to support real-time synchronization and cloud connectivity. Machine learning models were trained and evaluated using both local environments and Google Colab with GPU acceleration support for deep learning experimentation and hyperparameter tuning.

5.3. Software Description

This section describes the major modules of the Speech-to-Text Predictor system. The system is designed in a modular way so that each part performs a specific function. The interaction between these modules is represented through the class diagram shown below.

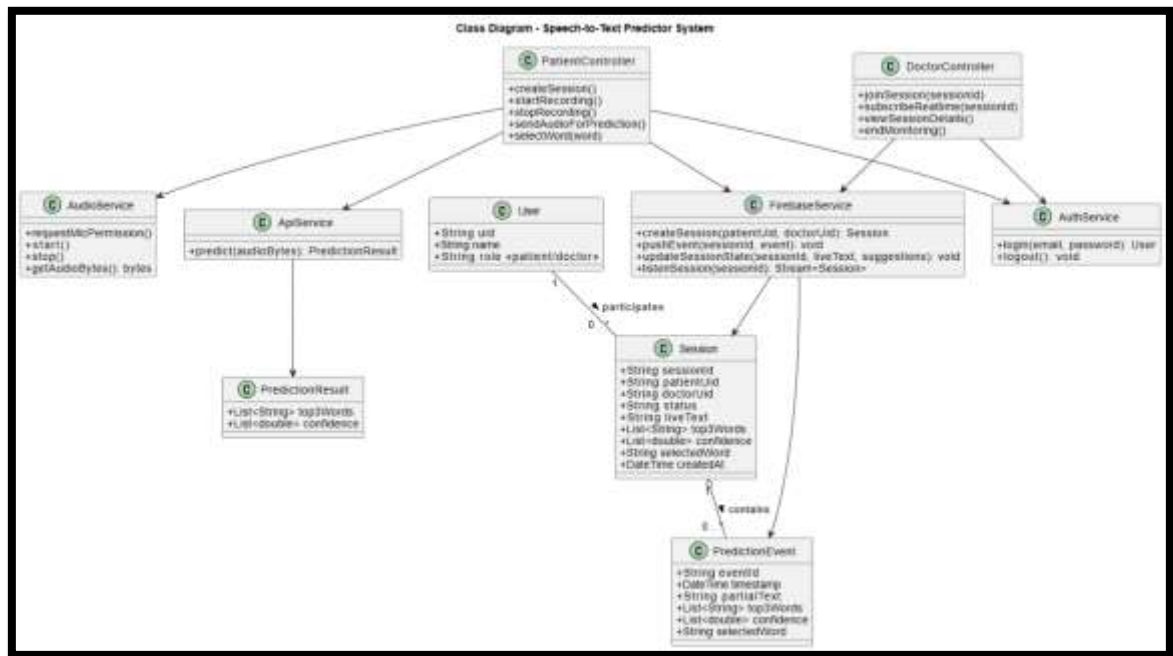


Figure5.1 Class Diagram

The class diagram illustrates the relationships between controllers, services, and core data entities such as PatientController, DoctorController, AudioService, ApiService, FirebaseService, AuthService, User, Session, PredictionResult, and PredictionEvent. These components collectively implement the functional workflow of the system.

The major software modules are described below.

i. User Interaction and Session Control

This module manages the interaction between patients and doctors during a speech session. The PatientController initiates sessions, starts and stops speech recording, and sends audio data for prediction. The DoctorController allows doctors to join sessions, receive live updates, and monitor speech progress. The Session class stores information about ongoing interactions, including transcripts, predicted words, confidence scores, and session status.

ii. Speech Capture and Audio Handling

The speech capture module is responsible for recording voice input from the patient. The AudioService class handles microphone permissions, audio recording, and

conversion of speech into digital byte streams. This audio data forms the input for subsequent processing and model prediction.

iii. Feature Extraction and Machine Learning Processing

In this module, raw speech signals are transformed into numerical representations suitable for machine learning models. Initially, preprocessing techniques such as noise reduction, normalization, and silence trimming are applied to enhance audio quality, and all audio is standardized to a fixed format (e.g., 16 kHz) to ensure consistency. Instead of relying only on basic MFCC features, the system utilizes a comprehensive 162 feature extraction pipeline, which combines multiple audio descriptors including Mel Spectrogram (128 features), MFCC (20 features), Chroma features (12 features), Spectral Centroid, and Zero Crossing Rate. This combined feature representation captures both spectral, temporal, and harmonic characteristics of speech, enabling the model to more effectively identify different stuttering patterns such as word repetitions, prolongations, and blocks.

Furthermore, the system supports real-time processing, where incoming speech is divided into small audio chunks (approximately 1–1.5 seconds) and sent to a Python based backend via RESTful APIs for immediate inference. The trained machine learning model processes these feature representations and return prediction results, including the detected stutter class and associated probability scores. The Flutter based ApiService communicates with backend endpoints (e.g., /predict and /suggest) using JSON-formatted requests and responses, and the results are used to dynamically update the user interface. Additionally, a hybrid detection approach is implemented by combining text based pattern recognition with audio based machine learning predictions, ensuring reliable detection even when speech-to-text processing normalizes or removes disfluencies. The final output of this module includes the detected stutter type, confidence scores, and context aware word suggestions, enabling accurate and real-time assistive support during live speech.

iv. Real Time Cloud Synchronization

This module ensures real-time sharing of speech data and predictions between patient and doctor applications. The FirebaseService manages the creation of sessions, updates

prediction events, and synchronizes transcripts and suggested words using Firebase Realtime Database. PredictionEvent records real-time speech events during a session, enabling continuous monitoring and interaction.

v. User Authentication and Role Management

The authentication module manages secure access to the system. The AuthService class verifies user credentials through Firebase Authentication and assigns roles (patient or doctor). The User class represents system users and stores essential information such as user identity and role.

5.3.1. Snippet 1: Patient Mobile Application Module

Covered Screens

- Welcome / Microphone Access
- Speaker Screen

Related User Stories

- E7-US1: Enable speech input
- E6-US1: Generate word predictions (via UI)

Snippet 1 – Microphone Permission Initialization (E7-US1)

```
Future<void> requestMicrophonePermission() async {  
  
    PermissionStatus status = await Permission.microphone.request();  
  
    if (status.isGranted) {  
  
        setState(() => micEnabled = true);  
  
    }  
  
}
```

Description: This function ensures microphone access is granted before enabling speech capture, which is critical for real-time speech processing.

Test Case (Patient App Initialization)

Table 5-1 Patient App Initialization

Test ID: E7-US1-TC1	E7-US1
Test Case Description: Verify microphone permission	
Inputs: User taps Enable Microphone	
Expected Result: Permission granted	
Actual Result: Permission granted	

5.3.2. Snippet 2: Real-Time Session Management Module

(FRONT END)

```
String generateSessionCode() {  
    final random = Random();  
    return 'AB-${100 + random.nextInt(900)}';  
}
```

Description: This logic generates a readable session identifier to establish synchronized communication.

Test Case:

Table 5-2 Session Creation

Test ID: E9-US1-TC1	US ID: E9-US1
Expected Result: Session code generated	
Actual Result: Session code generated	

5.3.3. Snippet 3 – Recording State Control

(FRONT END)

```
void toggleRecording(bool value) {  
    setState(() {  
        isRecording = value;  
        statusText = value ? "Recording" : "Idle";  
    });  
}
```

Description: This function manages the speech capture lifecycle.

Test Case:

Table 5-3 Recording Toggle

Test ID: E6-US1-TC1	US ID: E6-US1
Expected Result: Recording state changes	
Actual Result: Recording state changes	

5.3.4. Snippet 4 – Live Connection Indicator

Doctor Monitoring Interface Module (Epic E8)

(FRONT END)

```
Widget connectionStatus(bool connected) {  
    return Icon(  
        Icons.circle,  
        color: connected ? Colors.green : Colors.red,  
        size: 12,  
    );  
}
```

```
}
```

Description: This visual indicator reflects real-time synchronization status.

Test Case:

Table 5-4 Monitoring View

Test ID: E8-US1-TC1	US ID: E8-US1
Expected Result: Live data displayed	
Actual Result: Live data displayed	

5.3.5. Snippet 5 (User Story: E2–US5) Audio Standardization

(BACK END)

```
df["Class"] = df.apply(get_class_from_votes, axis=1)
```

```
df = df.dropna(subset=["Class"]).copy()
```

```
START_STOP_ARE_SECONDS = detect_seconds_or_samples(df)
```

```
for _, row in df.iterrows():
```

```
    audio_path = os.path.join(AUDIO_DIR, f'{row["EpId"]}.wav')
```

```
    if not os.path.exists(audio_path):
```

```
        continue
```

```
    info = sf.info(audio_path)
```

```
    if info.samplerate != SAMPLE_RATE:
```

```
        continue
```

```

s = int(float(row["Start"]) * info.samplerate) if START_STOP_ARE_SECONDS else
int(row["Start"])

e = int(float(row["Stop"]) * info.samplerate) if START_STOP_ARE_SECONDS else
int(row["Stop"])

s = max(0, s)

e = min(info.frames, e)

if e <= s:

    continue

clip_audio, _ = sf.read(audio_path, start=s, frames=e - s, dtype="float32")

if clip_audio.ndim == 2:

    clip_audio = clip_audio.mean(axis=1)

out_dir = os.path.join(OUT_DIR, safe_name(row["Class"]), safe_name(row["EpId"]))

os.makedirs(out_dir, exist_ok=True)

sf.write(os.path.join(out_dir, f"{row['ClipId']}.wav"), clip_audio, SAMPLE_RATE)

```

TestCase:

Table 5-5 Audio Standardization

TC ID	Input (Sample Rate)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US2-TC1	16000 Hz	0–16000 Hz	Audio remains unchanged	Pass
E2-US2-TC2	44100 Hz	>16000 Hz	Audio resampled to 16000 Hz	Pass

TC ID	Input (Sample Rate)	ECP (Range)	Expected Output	Actual Result (Pass/Fail)
E2-US2-TC3	8000 Hz	<16000 Hz	Audio resampled to 16000 Hz	Pass
E2-US2-TC4	0 Hz	Invalid	Audio rejected as invalid	Pass
E2-US2-TC5	Corrupted audio file	Valid / Corrupt	Standardization fails and error logged	Fail

Description: This preprocessing module converts dataset annotations into class-wise segmented audio clips. It first assigns a single label to each sample using majority voting, validates essential columns, and automatically generates missing ClipIds. The pipeline enforces a consistent sample rate (16 kHz), safely converts Start/Stop boundaries into valid sample indices, extracts only the required audio segment, converts stereo signals to mono, and writes each processed clip into class-wise folders. To ensure reliability, missing audio files, invalid boundaries, and read failures are skipped and logged without stopping the full preprocessing run.

5.3.6. Snippet 7 (E2–US7) Class Balancing of Processed Dataset

(BACK END)

```
SRC = "data/processed_class"
```

```
OUT = "data/balanced_classes_3200"
```

```
TARGET = 3200
```

```
os.makedirs(OUT, exist_ok=True)
```

```
for cls in os.listdir(SRC):
```

```
    cls_path = os.path.join(SRC, cls)
```

```
    files = [f for f in os.listdir(cls_path) if f.endswith(".wav")]
```

```
    os.makedirs(os.path.join(OUT, cls), exist_ok=True)
```

```

selected = random.sample(files, TARGET) if len(files) >= TARGET \

    else files + random.choices(files, k=TARGET-len(files))

for f in selected:

    shutil.copy(os.path.join(cls_path, f), os.path.join(OUT, cls, f))

print("Balanced dataset created.")

```

Description: This code balances the speech dataset by ensuring that each class contains exactly 3200 audio samples. For classes with more than the target number of samples, random sampling is applied, while classes with fewer samples are expanded through controlled duplication. The balanced audio files are copied into a separate output directory, preserving the original dataset and class labels for unbiased model training.

Test Case:

Table 5-6 Class Balancing

TC ID	Input	ECP	Expected Output	Actual Output
E2– US7– TC1	Class contains >3200 samples	3200>0	System randomly selects samples to reduce class size to 3200	Pass
E2– US7– TC2	A speech class contains exactly 3200 samples	Classes=320 0	Class remains unchanged with 3200 samples	Pass
E2– US7– TC3	A speech class contains fewer than 3200 samples	3200<0	System duplicates samples to reach 3200	Pass

5.3.7. Snippet 8 Advanced Feature Extraction & Audio Standardization (Epic E3)**Code:**

```
def extract_215_features(path):
    y, sr = librosa.load(path, sr=SR, mono=True)

    # Remove leading/trailing silence
    y, _ = librosa.effects.trim(y, top_db=25)

    # Normalize amplitude
    if np.max(np.abs(y)) > 0:
        y = y / np.max(np.abs(y))

    # Fix audio length to 3 seconds
    y = librosa.util.fix_length(y, size=SR * DURATION)

    mel = librosa.feature.melspectrogram(
        y=y, sr=SR, n_mels=N_MELS
    )
    mel = librosa.power_to_db(mel, ref=np.max)

    mfcc = librosa.feature.mfcc(
        y=y, sr=SR, n_mfcc=N_MFCC
    )
    mfcc_delta = librosa.feature.delta(mfcc)
    mfcc_delta2 = librosa.feature.delta(mfcc, order=2)

    chroma = librosa.feature.chroma_stft(y=y, sr=SR)
    centroid = librosa.feature.spectral_centroid(y=y, sr=SR)
    bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=SR)
    rolloff = librosa.feature.spectral_rolloff(y=y, sr=SR)
    contrast = librosa.feature.spectral_contrast(y=y, sr=SR)
    flatness = librosa.feature.spectral_flatness(y=y)
    rms = librosa.feature.rms(y=y)
    zcr = librosa.feature.zero_crossing_rate(y)
    onset = librosa.onset.onset_strength(y=y, sr=SR).reshape(1, -1)

    silence_mask = (rms < np.mean(rms)).astype(np.float32)
```

```

features = [
    mel, mfcc, mfcc_delta, mfcc_delta2, chroma,
    centroid, bandwidth, rolloff, contrast, flatness,
    rms, zcr, onset, silence_mask
]

```

```

features = [fix_frames(f) for f in features]
combined = np.vstack(features)

```

```

return combined.astype(np.float32)

```

Description: This module performs feature extraction on preprocessed audio clips for stutter classification. Each audio sample is standardized to 16 kHz and fixed to a 3-second duration to maintain consistent input size. The module extracts important acoustic features including Mel Spectrogram, MFCC, Delta features, Chroma, spectral features, RMS, ZCR, onset strength, and silence mask. All features are combined into a final feature matrix of shape (215, 94), which is then used as input for the CNN model.

Test Case (Feature Extraction)

Table 5-8 Speech Feature Extraction

Test ID	US ID	Test Case Description	Inputs	Expected Result	Actual Result
E3-US6-TC1	E3-US6	Audio length standardization	1.5s audio clip	Padded to exactly 3.0s	PASSED
E3-US7-TC1	E3-US7	162-Feature Combo Extraction	Valid standardized clip	Array shape: (162, 94)	PASSED
E3-US7-TC3	E3-US7	Exception handling for corrupt files	Corrupt .wav file	Error logged; skip file	PASSED

5.3.8. Snippet 9 Class-wise Feature Aggregation & Storage (Epic E3)

Code:

```
def main():

    classes = ["Block", "Interjection", "NaturalPause", "Prolongation", "WordRep"]

    for cls in classes:

        all_features = []

        for f in tqdm(files, desc=f"Extracting {cls}"):

            feat = extract_all_features(f)

            if feat is not None:

                all_features.append(feat)

        if all_features:

            final_array = np.array(all_features)

            save_path = OUT_DIR / f"{cls}_zcr_v2_X.npy"

            np.save(save_path, final_array)

        del all_features

        gc.collect() # Clear memory
```

Description: After feature extraction, this module aggregates all features for each stutter class (e.g., Block, Prolongation, etc.). These aggregated features are saved in a memory-efficient. npy format to enable fast data loading during model training. During processing, garbage collection (gc.collect()) is used to clear memory and prevent RAM overflow.

Test Case:

Table 5-9 Class-wise Feature Aggregations

Test ID	US ID	Test Case Description	Inputs	Expected Result	Actual Result
E3-US8-TC1	E3-US8	Bulk feature aggregation	500 clips of "Block"	Single .npy file created	PASSED
E3-US8-TC3	E3-US8	Memory optimization check	Processing 5 classes	RAM usage remains stable	PASSED

5.3.9. Snippet 10 Anti-Overfitting CNN Architecture (Epic E4)

```
def build_lightweight_cnn(input_shape, num_classes):  
    model = models.Sequential([  
        layers.Input(shape=input_shape),  
  
        layers.Conv2D(16, (3, 3), activation="relu", padding="same"),  
        layers.MaxPooling2D((2, 2)),  
  
        layers.Conv2D(32, (3, 3), activation="relu", padding="same"),  
        layers.MaxPooling2D((2, 2)),  
        layers.Dropout(0.25),  
  
        layers.Conv2D(64, (3, 3), activation="relu", padding="same"),  
        layers.MaxPooling2D((2, 2)),  
        layers.Dropout(0.30),  
  
        layers.GlobalAveragePooling2D(),  
  
        layers.Dense(128, activation="relu"),  
        layers.Dropout(0.40),  
  
        layers.Dense(num_classes, activation="softmax")  
    ])  
  
    model.compile(
```

```
optimizer=tf.keras.optimizers.Adam(learning_rate=5e-5),
loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)

return model

model = build_lightweight_cnn(input_shape, num_classes)

callbacks = [
tf.keras.callbacks.EarlyStopping(
monitor="val_loss",
patience=5,
restore_best_weights=True
),
tf.keras.callbacks.ReduceLROnPlateau(
monitor="val_loss",
factor=0.5,
patience=3
)
]

history = model.fit(
X_train,
y_train,
validation_data=(X_val, y_val),
epochs=20,
batch_size=32,
callbacks=callbacks
)

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
```

Description: This module trains the CNN model using the extracted 215-feature matrices as input. The feature data is reshaped into (215, 94, 1) format and passed through convolution, pooling, dropout, and dense layers for stutter classification. The model is trained using the Adam optimizer with validation monitoring and learning-rate adjustment to improve

performance. After training and evaluation, the model achieved 94.34% accuracy with a 0.92 macro F1-score.

TestCase:

Table 5-10 Anti overfitting

Test ID	US ID	Test Case Description	Inputs	Expected Result	Actual Result
E4-US2-TC1	E4-US2	Verify CNN compilation with regularizers	Input shape: (162, 94, 1)	Model compiles without errors	PASSED

5.3.10. Snippet 11 Model Training & Evaluation (Epic E4)

Code:

Training process with Checkpoint

```
ckpt = tf.keras.callbacks.ModelCheckpoint('stutter_robust_final.keras',  
monitor='val_accuracy', save_best_only=True, verbose=1)
```

```
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=40,  
batch_size=32, callbacks=[ckpt])
```

```
model.load_weights('stutter_robust_final.keras')
```

```
y_pred = np.argmax(model.predict(X_test), axis=1)
```

```
print(classification_report(y_test, y_pred, target_names=CLASSES))
```

Description: During the training phase, the model is trained with a low learning rate (0.00003) to ensure smooth convergence. The ModelCheckpoint callback saves only the best-performing weights based on validation accuracy. Finally, a strict classification report is generated on unseen test data (20%) to measure true precision and recall.

Test Case:

Table 5-11 Model Training

Test ID	US ID	Test Case Description	Inputs	Expected Result	Actual Result
E4-US3-TC1	E4-US3	Execute training with checkpoint saving	X_train, y_train, 40 Epochs	.keras file saved on best val_acc	PASSED
E4-US3-TC3	E4-US3	Generate final classification report	X_test, y_test arrays	Precision/Recall output printed	PASSED

5.3.11. Snippet 12 – Real-Time Prediction Trigger & Suggestion Engine (Epic E6)

```
void _loadSuggestions(String text) async {  
    final prefix = _cleanPrefix(text);  
  
    setState(() {  
        currentPredictions = ["..."];  
    });  
    try {  
        final suggestions =  
            await DetectionService.getSuggestions(prefix);  
        if (!mounted) return;  
        setState(() {  
            currentPredictions = suggestions.isNotEmpty  
                ? suggestions.take(5).toList()  
                : [];  
        });  
    } catch (e) {  
        // Handle error  
    }  
}
```

```
        : ["go", "speak", "help"];

    });

} catch (e) {

    setState(() {

        currentPredictions = ["go", "speak", "help"];

    });

}}

void _appendPrediction(String word) {

    final current = transcriptCtrl.text.trim();

    setState(() {

        transcriptCtrl.text =

            current.replaceAll("___", word);

        currentPredictions = [];

        isAwaitingSelection = false;

        stutterStatus = "Listening...";

    });

    _startListening();

}
```

Description:

This module is responsible for activating the prediction engine whenever a stuttering event is detected during live speech. The prediction process is integrated with the real-time stutter detection module and operates only when disfluency patterns such as Word Repetition, Prolongation, or Block are identified. Once a stutter is detected, the system temporarily pauses continuous listening, inserts a blank placeholder (___) into the transcript, and extracts the current speech prefix to generate contextual predictions. The Flutter application communicates

with the backend prediction API using RESTful requests. The extracted speech context is sent to the backend, which processes the input and returns ranked word suggestions in JSON format. These suggestions are then displayed dynamically on the interface as tappable prediction chips. When the user selects a suggestion, the selected word replaces the blank placeholder in the transcript, and the system automatically resumes continuous speech monitoring. To improve responsiveness and prevent repeated API calls during rapid speech events, debounce handling and asynchronous processing are implemented. Additionally, fallback suggestions are displayed whenever the backend response fails or no reliable predictions are returned. This ensures stable and uninterrupted user interaction during real-time communication sessions. The implemented prediction engine combines context-aware NLP-based prediction with real-time stutter detection to provide assistive communication support for individuals who stutter.

Test Case:

Table 5-12 Model Prediction

TC ID	Input	ECP (Range)	Expected Output	Actual Output	Status
E6-US1-TC1	Stutter detected as WordRep	Stutter Flag = 1	Prediction module activates automatically	Prediction trigger activated and suggestion loading started successfully	Pass
E6-US1-TC2	Stutter detected as Prolongation	Stutter Flag = 1	Prediction module activates automatically	Prediction engine activated for detected stutter state	Pass
E6-US1-TC3	Fluent speech detected	Stutter Flag = 0	Prediction module does not activate	Prediction trigger remained inactive during fluent speech	Pass

E6-US2-TC1	Valid speech prefix entered	Prefix = Valid	Context-related word suggestions are generated	Suggestions were fetched and displayed as tappable chips	Pass
E6-US2-TC2	Backend/API returns no suggestions	Suggestions = 0	Fallback suggestions are displayed	Default fallback suggestions were displayed successfully	Pass
E6-US2-TC3	User selects prediction word	Selection = 1	Selected word replaces blank placeholder and listening resumes	Selected word replaced the placeholder and listening restarted successfully	Pass

5.3.12. Snippet 13 – Firebase Real-Time Synchronization Module (Epic E9)

Code:

```
import 'package:firebase_database/firebase_database.dart';

class FirebaseSessionService {

  static final DatabaseReference _db = FirebaseDatabase.instance.ref();

  static const String sessionId = "session_001";

  static Future<void> updateSession({
    required String transcript,
    required String status,
    required List<String> predictions,
    required bool isRecording,
    required bool isAwaitingSelection,
  }) async {
```

```
await _db.child("sessions/$sessionId").set({  
    "transcript": transcript,  
    "status": status,  
    "predictions": predictions,  
    "isRecording": isRecording,  
    "isAwaitingSelection": isAwaitingSelection,  
    "timestamp": ServerValue.timestamp,  
});  
}  
  
static Stream<DatabaseEvent> sessionStream() {  
    return _db.child("sessions/$sessionId").onValue;  
}  
}
```

Description: This module is responsible for synchronizing live speech session data between the patient and doctor/monitor interfaces using Firebase Realtime Database. The `updateSession()` method stores the current transcript, stutter detection status, prediction suggestions, recording state, and selection state under the active session node. Whenever the patient speaks, selects a prediction, starts recording, or stops recording, the latest state is written to Firebase. The `sessionStream()` method provides a continuous Firebase listener for the monitor interface. It listens to the same session node and updates the monitor screen whenever new data is received. Through this implementation, the doctor/monitor side can view the patient's live transcript, detected stutter status, and context-related prediction suggestions in real time. This Firebase synchronization module supports the real-time communication requirement of the system by ensuring that both patient and monitor screens remain consistent during an active session. It also improves system usability by allowing the monitor interface to reload the latest session state after reconnection.

Test Case:

Table 5-13 Firebase Synchronization

TC ID	Input	ECP (Range)	Expected Output	Actual Output	Status
EP9-US1-TC1	Live transcript update	Session Active = True	Transcript instantly synced to monitor interface	Transcript successfully synchronized between patient and monitor screens in real time	Pass
EP9-US1-TC2	Stutter detected as WordRep	Detection State = Active	Same stutter status displayed on both devices	WordRep status displayed correctly on both interfaces	Pass
EP9-US1-TC3	Recording stopped manually	Recording Status = False	Both interfaces show recording paused	Recording state updated successfully across connected devices	Pass
EP9-US2-TC1	Context-related prediction suggestions generated	Prediction Count = 0–5	Same prediction chips visible on monitor screen	Prediction suggestions synchronized successfully in real time	Pass
EP9-US2-TC2	Patient selects one prediction word	Selected Prediction = 1	Selected word instantly synced to both interfaces	Selected prediction updated correctly on monitor interface	Pass

5.4. Implemented UI

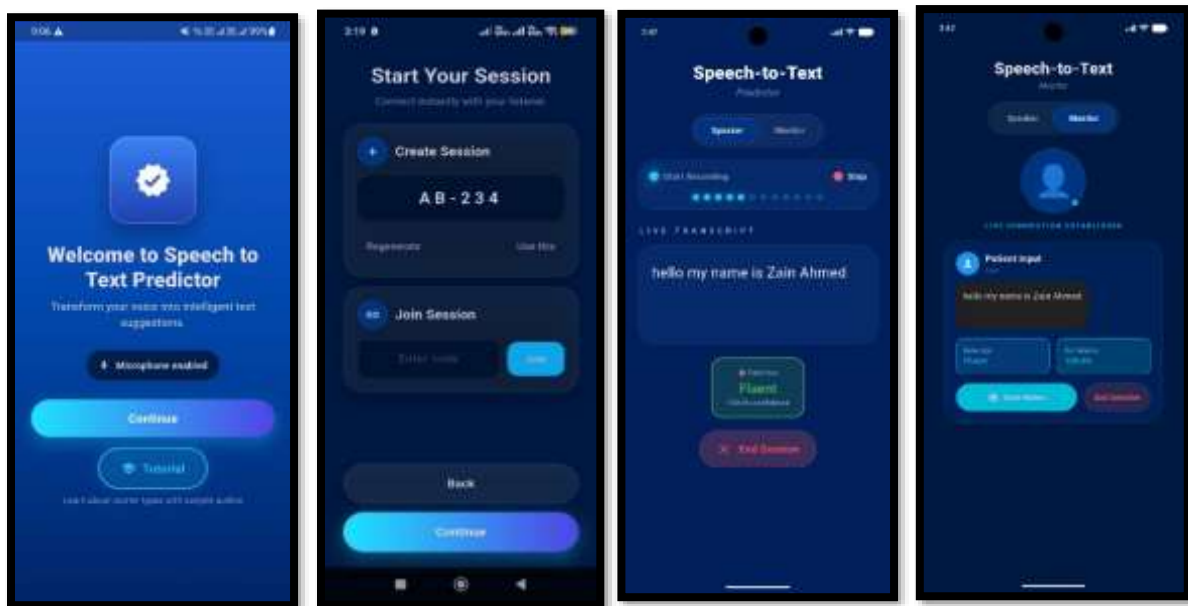


Figure 5.4 Implemented UI

5.4.1. Interjection

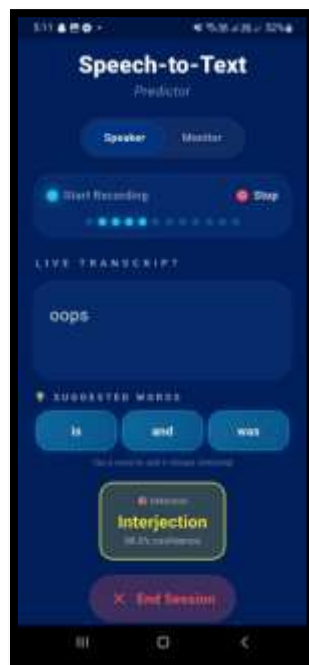


Figure 5.4.1 Interjection

Interjection is detected as sudden uhh, ahhh sounds or oops etc.

5.4.2. Block



Figure 5.4.2 Block

Block is type of stutter that is detected from a sudden break in the sentence.

5.4.3. Prolongation

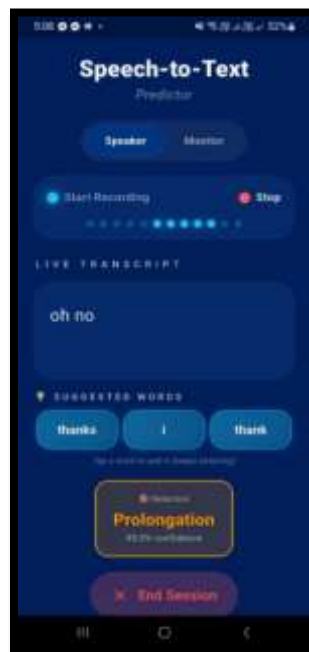


Figure 5.4.3 Prolongation

Prolongation is a type of stutter that is detected through the stretch of word while speaking like here, it detected wowww...

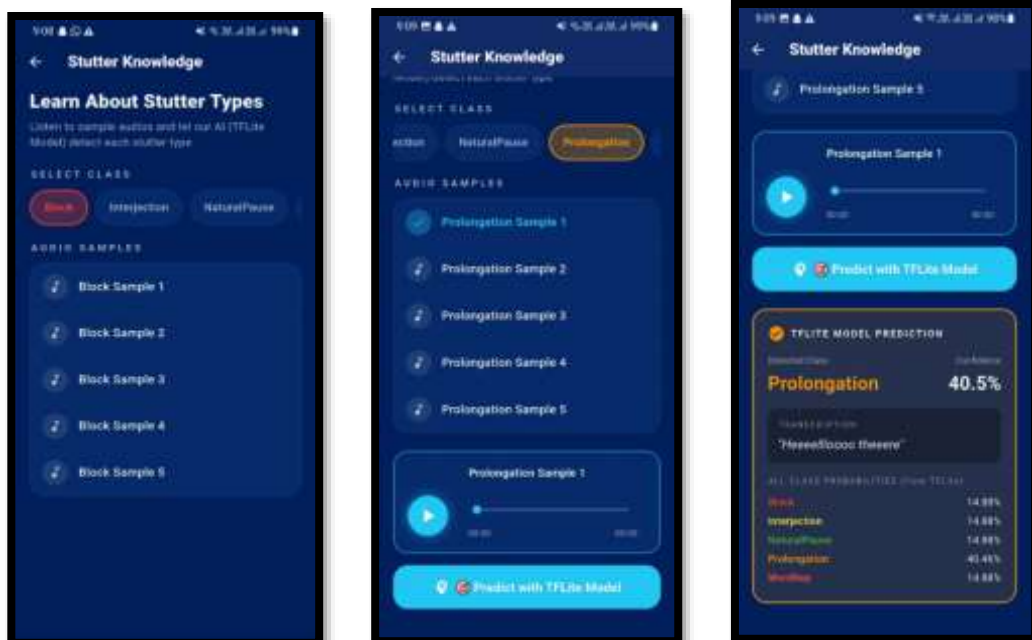
5.4.4. WordRep



Figure 5.4.4 WordRep

5.4.5. Testing Screen

Figure 5.4.5 Testing Screens



In this screen, all the classes are tested under our trained dataset and it gives the confidence score listening to it.

Chapter 6

6. Software Deployment

This chapter presents the complete deployment process of the Speech to Text Predictor system, covering all key stages from model training and mobile application development to build generation and cloud backend configuration. The system has been successfully implemented and tested on both a physical Android device (Samsung SM-A325F) and a virtual Pixel 9 Pro XL emulator. The deployment architecture is cloud-based, with Firebase serving as the real-time backend and the Flutter-based mobile application deployed on Android devices. The machine learning component has been fully trained, validated, and integrated into the application. The following sections document each deployment phase in detail, supported by evidence-based screenshots from the actual development environment.

6.1. Installation / Deployment Process Description

The deployment of the Speech to Text Predictor system follows a structured, phased approach aligned with the project's development lifecycle. The process encompasses four key pillars: machine learning model training, mobile application development and build generation, cloud backend configuration via Firebase, and end-to-end functional testing on both physical and emulated Android hardware. Each phase is described below alongside corresponding evidence from the actual implementation.

Figure 6.1 illustrates the overall deployment architecture of the system, showing the relationships between the development environment, the trained machine learning model, the Firebase cloud backend, and the two Flutter-based mobile applications.

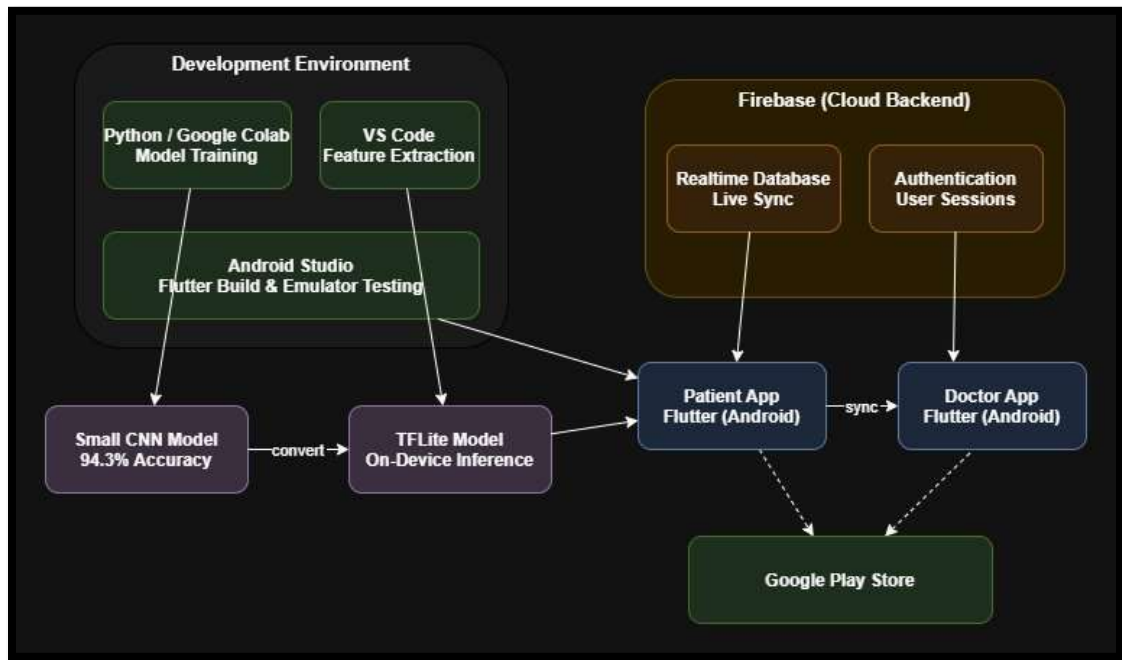


Figure 6.1: Deployment Architecture of Speech to Text Predictor System

6.2. Machine Learning Model Training and Evaluation

The machine learning backbone of the Speech to Text Predictor is a Small CNN model trained to classify five distinct stuttering event types: Block, Interjection, NaturalPause, Prolongation, and WordRepetition. The model was trained in Python using Google Colab, leveraging GPU acceleration for efficient deep learning experimentation.

The training pipeline involved speech preprocessing, extraction of per-class acoustic features stored as NumPy arrays with an input shape of $215 \times 94 \times 1$ per sample, min-max normalization applied independently to each class, and class balancing to address data distribution imbalances. Specifically, the Block, Interjection, Prolongation, and WordRep classes were oversampled to reach a target of 5,000 training samples per class, while the NaturalPause class — which had the largest original sample count of 10,124 — was undersampled to match the same target. This ensured a balanced training distribution of 25,000 total samples across all five classes.

The CNN architecture comprised three Conv2D layers (with 16, 32, and 64 filters respectively), each followed by MaxPooling2D and incremental Dropout layers (0.25, 0.30) to mitigate overfitting, succeeded by a GlobalAveragePooling2D layer, a Dense layer of 128 units with 0.40 Dropout, and a final softmax output layer. The model was compiled with the Adam optimizer at a learning rate of 5×10^{-5} and trained over 26 epochs with a batch size of 32.

As illustrated in Figure 6.2, the model achieved an overall accuracy of 94.3% on the held-out test set of 3,413 samples, with a weighted average F1-score of 0.94 and a macro-average F1-score of 0.92. The Interjection, NaturalPause, and Prolongation classes achieved near-perfect precision and recall scores of 1.00, demonstrating strong discriminative ability for the majority of stutter types. The confusion matrix confirms that residual misclassifications are largely confined to the Block and WordRep classes, which are acoustically more similar to natural speech patterns. The training report was saved to Google Drive for reproducibility and future reference.

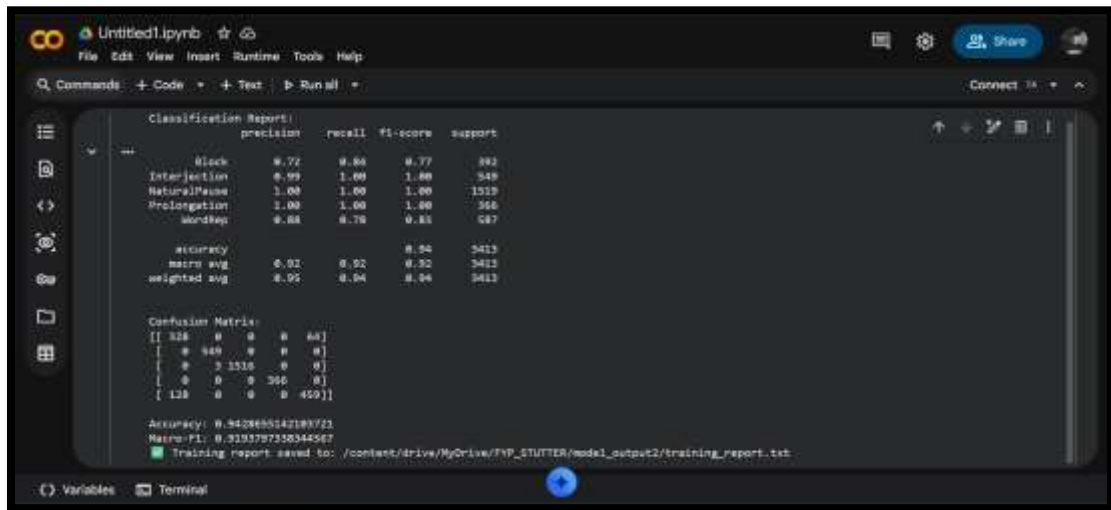


Figure 6.2: Small CNN Model – Classification Report and Confusion Matrix (Google Colab, 94.3% Accuracy)

The trained model was subsequently exported in TensorFlow Lite (TFLite) format to enable efficient on-device inference within the Flutter mobile application, ensuring low-latency real-time stutter detection without dependence on external compute resources.

6.3. Firebase Realtime Database Configuration

Firebase serves as the cloud backend for the Speech to Text Predictor, enabling real-time data synchronization between the patient-facing and doctor-facing interfaces. The Firebase project, titled “Speech To Text Predictor,” was configured within the Firebase Console under the project identifier `speech-to-text-predictor`, hosted on the Firebase Realtime Database at the endpoint:

`https://speech-to-text-predictor-default-rtdb.firebaseio.com`

Figure 6.3 presents a live view of the Firebase Realtime Database during an active session. The database structure captures session-level data in real time, including the following fields:

- **confidence:** A numerical score (e.g., 0.92) representing the model’s certainty in the stutter classification.
- **isAwaitingSelection:** A boolean flag indicating whether the doctor monitor is awaiting word selection from the patient.
- **isRecording:** A real-time indicator reflecting the active recording state of the patient device.
- **predictions:** An array of context-aware word suggestions generated by the word prediction engine.
- **status:** The detected stutter type (e.g., “WordRep”), updated in real time upon each detection event.
- **transcript:** The live speech transcript as recognized by the application (e.g., “Hi my (name name) is”).
- **updatedAt:** A Unix timestamp recording the time of the last database update.

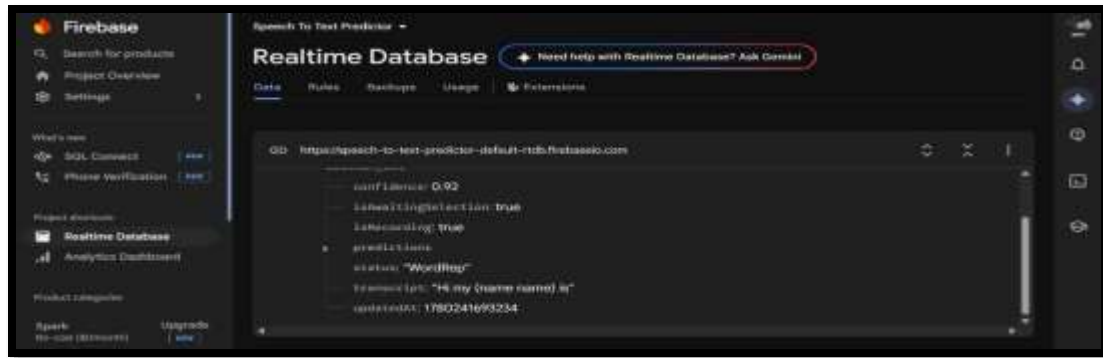


Figure 6.3: Firebase Realtime Database – Live Session Data for Speech To Text Predictor

This real-time synchronization architecture ensures that both the patient and the doctor/monitor devices receive identical session data instantaneously, enabling seamless collaborative communication support without perceptible latency.

6.4. Mobile Application Build and APK Generation

The Flutter-based mobile application was developed and built using Android Studio. To prepare the application for deployment and distribution, both a debug and a release APK were generated via the Flutter build pipeline. The release APK was compiled in optimized mode, producing a significantly reduced binary size compared to the debug build, making it suitable for installation on end-user devices. Figure 6.4 shows the Android Studio build terminal confirming the successful completion of the release APK generation process. The Gradle task `assembleRelease` was executed successfully, and Flutter's tree-shaking optimization reduced the MaterialIcons font asset from 1,645,184 bytes to just 3,164 bytes — a 99.8% reduction — significantly decreasing the overall application footprint. The build completed in 12.8 seconds and exited with code 0, confirming a clean, error-free compilation.



Figure 6.4: Android Studio Terminal – Successful Release APK Build

Figure 6.5 presents the output directory of the build process, displaying both the debug and release APK files. The app-release.apk (71,230 KB) represents the production-ready build, while app-debug.apk (106,131 KB) was used during the development and testing phases. Both files include corresponding SHA1 hash files for integrity verification.

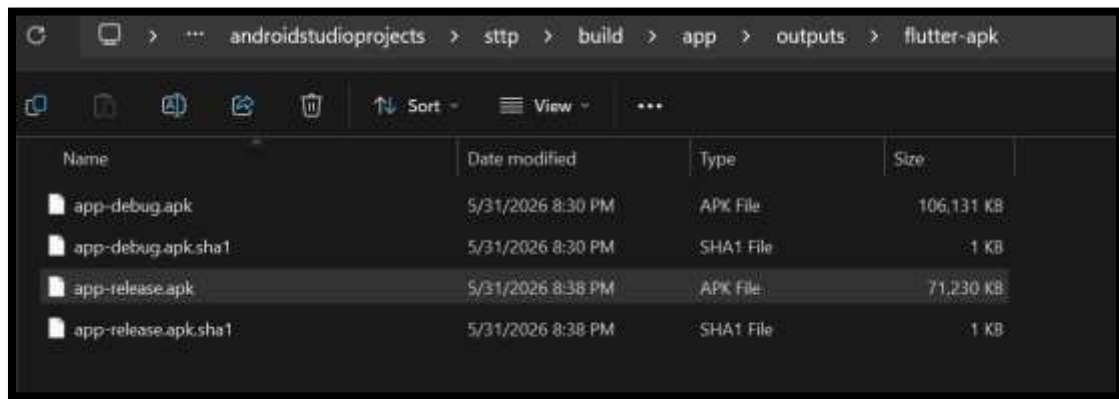


Figure 6.5: Flutter APK Output Directory – app-release.apk and app-debug.apk Files

The release APK was validated for installation on Android devices running API Level 33 (Android 13) and above, consistent with the target device specifications used throughout the testing phase.

6.5. Mobile Application – Patient Interface

The patient-facing interface of the Speech to Text Predictor provides a real-time speech interaction environment tailored for individuals who stutter. The application was tested on a Samsung SM-A325F physical device running Android API Level 33, as well as on a Pixel 9 Pro XL Android emulator, both connected to Android Studio for live debugging and validation.

As shown in Figure 6.6, the patient screen displays the following functional components during an active session:

- **Speaker / Monitor toggle:** Allows the user to switch between the patient (Speaker) and monitoring (Monitor) roles.
- **Start Recording / Stop controls:** Initiates and terminates the real-time audio capture pipeline.

- **Live Transcript panel:** Displays the ongoing speech-to-text output in real time (e.g., “Hi my (name name) is”).
- **Suggested Words:** Presents context-aware word predictions (e.g., “name,” “same,” “game”) to assist the speaker during moments of disfluency.
- **Stutter Detector badge:** Displays the detected stutter category (e.g., “WordRep”) alongside the model’s confidence score.
- **End Session button:** Terminates the current session and clears the active database entry.

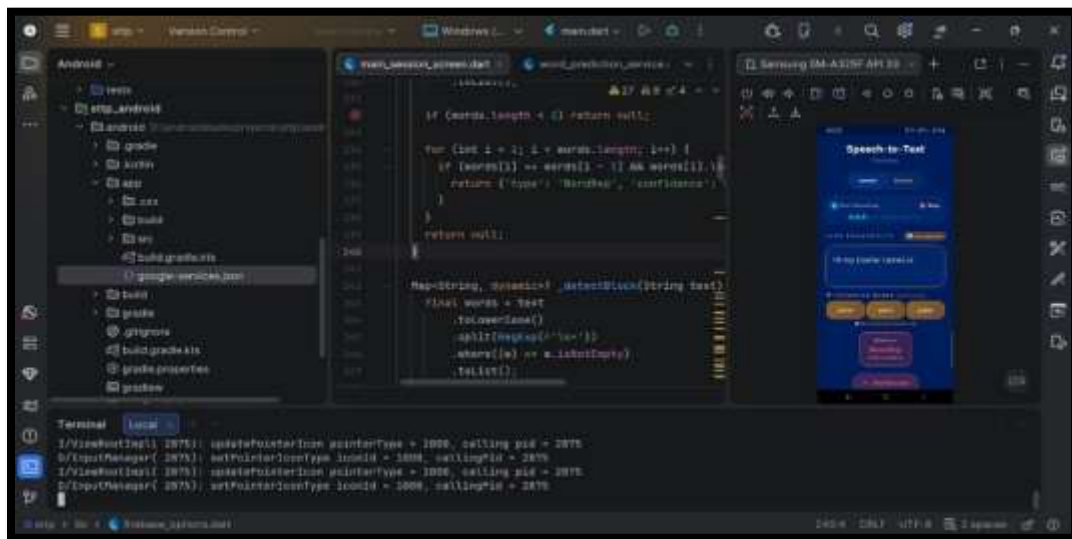


Figure 6.6: Patient Interface – Real-Time Speech Interaction Screen Running on Samsung SM-A325F (Android Studio)

The patient interface was validated against all defined user stories under Epic E7, confirming functional compliance with the requirements for real-time speech interaction and live prediction display. Testing was conducted on both the physical Samsung SM-A325F device and the Pixel 9 Pro XL emulator, with consistent behaviour observed across both environments.

6.6. Mobile Application – Doctor Monitor Interface

The doctor-facing monitor interface operates in parallel with the patient application, receiving synchronized session data via Firebase Realtime Database. The monitor screen is designed to give a clinician or communication partner a real-time view of the patient’s speech session, including detected stutter events, confidence scores, and word prediction suggestions.

Figure 6.7 illustrates the doctor monitor interface running on the Samsung SM-A325F device during an active session with the patient application. The interface displays:

- **Live Connection Established indicator:** Confirms an active real-time link between the patient and monitors devices.
- **Patient Input panel:** Shows the live transcript as received from the patient device (e.g., “Hi my (name name) is”).
- **Detection and Confidence metrics:** Displays the stutter type detected (e.g., WordRep) and the model’s confidence level (e.g., 90.0%).
- **Prediction Suggestions:** Mirrors the word predictions shown on the patient screen, providing the monitor with full contextual awareness.
- **Save Notes / End Session controls:** Allow the monitor to document clinical observations and terminate the session remotely.

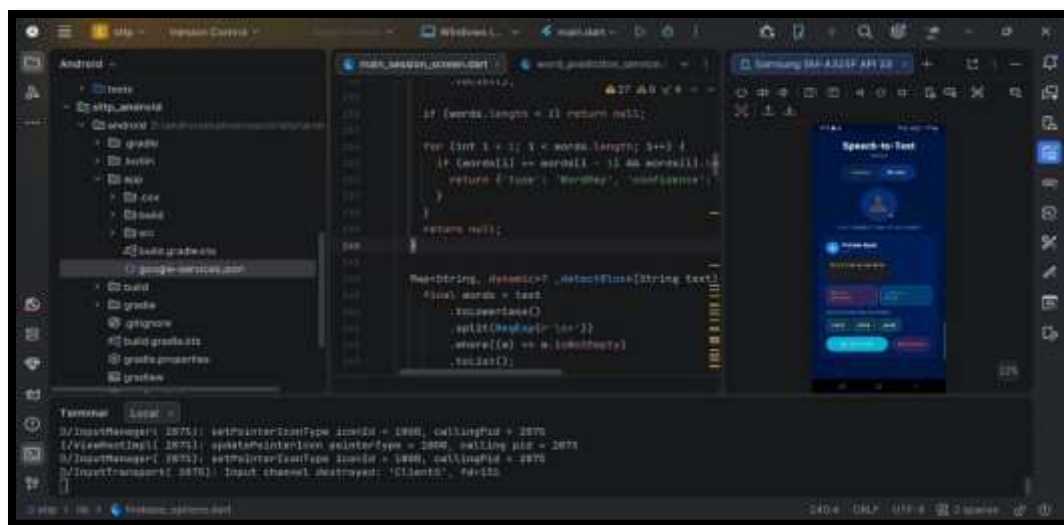


Figure 6.7: Doctor Monitor Interface – Real-Time Session View with Stutter Detection and Prediction Sync

The bidirectional synchronization between the patient and monitor interfaces was validated successfully on both the physical Samsung device and the Pixel 9 Pro XL emulator. All stutter events, confidence scores, transcripts, and word predictions appeared on the monitor screen within an imperceptible latency window, confirming the efficacy of the Firebase Realtime Database integration.

6.7. Deployment Summary

The Speech to Text Predictor has been successfully deployed as a fully functional Android application, with all core system components operational and validated. Table 6.1 provides a consolidated summary of the deployment status across each system layer.

Table 6.1: Deployment Status Summary – Speech to Text Predictor System

Component	Status	Details
Small CNN Model Training	Completed	94.3% accuracy, 26 epochs, saved to Google Drive
TFLite Model Integration	Completed	On-device inference, 215×94×1 input shape
Firebase-Realtime Database	Configured & Live	Real-time sync validated, 7 fields per session
Patient App (Flutter)	Deployed & Tested	Samsung SM-A325F API 33 and Pixel 9 Pro XL emulator
Doctor Monitor App (Flutter)	Deployed & Tested	Bidirectional sync confirmed on both devices

Release APK Generation	Completed	app-release.apk, 71.2 MB, exit code 0
Google Play Store Release	In progress	Signed AAB will be submitted after review

6.8. Current Deployment – Google Play Store

Upon completion of final testing, performance benchmarking, and security hardening, the Speech to Text Predictor application will be prepared for public release via the Google Play Store. This process will involve generating a signed Android App Bundle (AAB) using Android Studio's release signing configuration, uploading the bundle to the Google Play Console, and providing all required metadata including UI screenshots, application descriptions, content ratings, and privacy policy documentation.

We have successfully created our Google Play Console developer account and completed the initial setup phase. Our account has been registered under the name Zain Ahmed from Rawalpindi, Pakistan with contact email zainahmedsatti@gmail.com and phone +92-318-5877408. We have registered our app "STTP - Stutter to Fluent Speech" (package name: `com.zainahmed.speechtotext`) as a Telehealth/medical app (assistive technology for speech disorders), and indicated that we are undecided on monetization for now, keeping all earning options open. We have selected Android API 36.1 as our development platform and completed the developer verification requirements. Currently, we are in the App Registration Security Phase, working on generating the SHA-256 certificate fingerprint required by Google to authenticate our app for distribution. This security key is essential to complete the package name registration and proceed with app listing creation and submission.

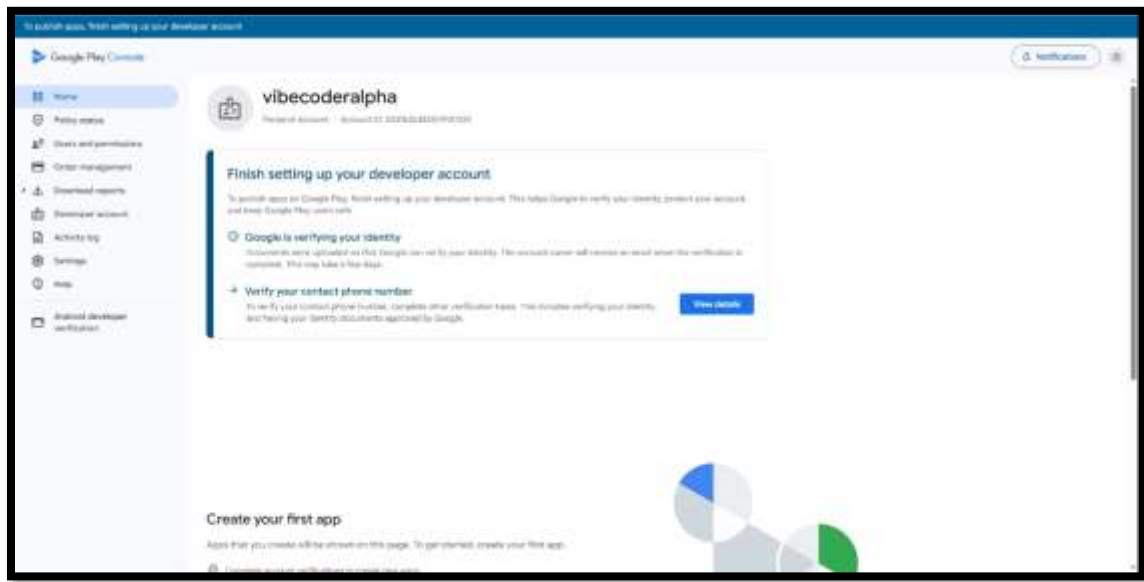


Figure 6.81- Google Playstore Console

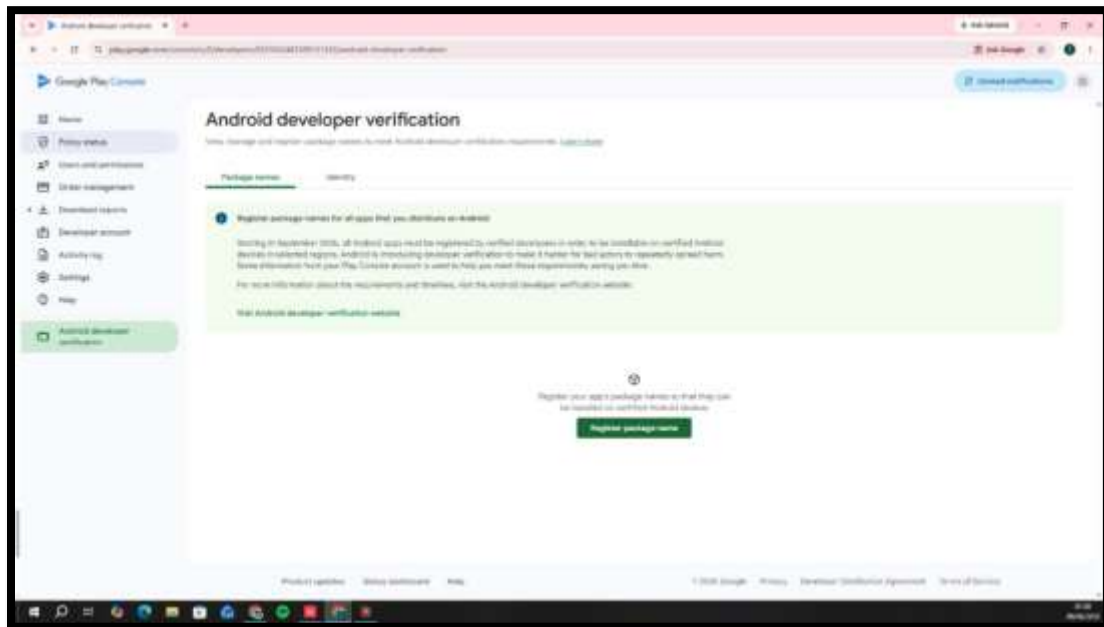


Figure 6.82- Google Playstore Account Verification

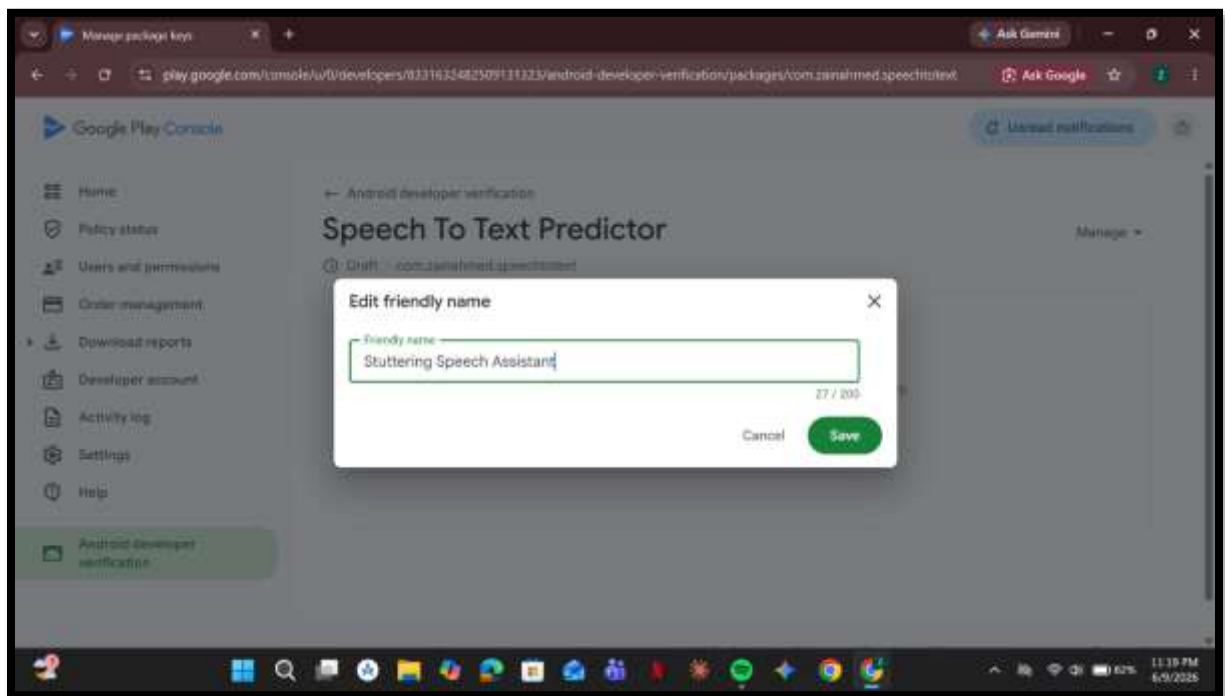


Figure 6.83- Friendly App name

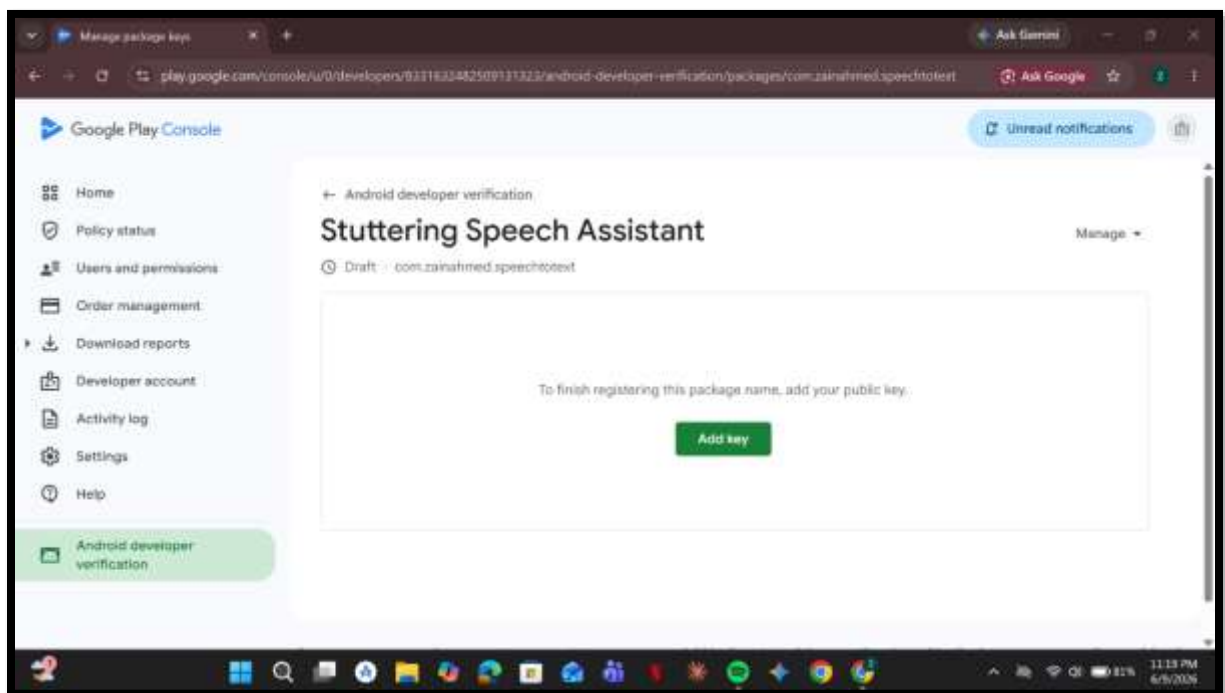


Figure 6.84- App name on App store

References

- [1] N. Alhakbani, R. Alnashwan, A. Al-Nafjan, and A. Almudhi, “*Automated Stuttering Detection Using Deep Learning Techniques*,” **Journal of Clinical Medicine**, vol. 14, no. 10, pp. 3552, May 2025. <https://doi.org/10.3390/jcm14103552>
- [2] R. V. Ramitha, R. Chainani, S. Mehrotra, S. Sah, and S. Mahajan, “*Evaluative comparison of machine learning algorithms for stutter detection and classification*,” **MethodsX**, vol. 13, no. (article 103050), pp. 103050, Nov. 2024. <https://doi.org/10.1016/j.mex.2024.103050>
- [3] E. Kourkounakis, R. Ranjan, and A. Papangelis, “*FluentNet: End-to-end stuttering detection using spectrograms, CNNs, and attention-based BiLSTMs*,” **Proc. INTERSPEECH**, pp. 1–5, Aug. 2021. Available: <https://arxiv.org/pdf/2009.11394>
- [4] S. A. Sheikh, M. Hussain, and M. Umair, “*StutterNet: A TDNN-based architecture for stuttering event detection*,” **IEEE Access**, vol. 9, pp. 1–10, 2021. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0925231222012772?via%3Dihub>
- [5] V. Prabhu and N. Seliya, “*Deep learning for stutter classification using the SEP-28K dataset*,” **Proc. IEEE Int. Conf. on Machine Learning and Applications (ICMLA)**, pp. 1–6, Dec. 2022. Doi: [10.1109/ICMLA55696.2022.00247](https://doi.org/10.1109/ICMLA55696.2022.00247)
- [6] S. Basak, A. Chatterjee, and R. Dey, “*TranStutter: Transformer-based stuttering detection from Mel-spectrograms*,” **Proc. IEEE Signal Processing Conference**, pp. 1–7, 2023. Available: <https://www.mdpi.com/1424-8220/23/19/8033>
- [7] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “*Advancing stuttering detection via data augmentation, class-balanced loss and multi-contextual deep learning*,” 2023. DOI:[10.1109/JBHI.2023.3248281](https://doi.org/10.1109/JBHI.2023.3248281)
- [8] Lea, C., Mitra, V., Joshi, A., Kajarekar, S., & Bigham, J. P. “*SEP-28k: A Dataset for Stuttering Event Detection From Podcasts With People Who Stutter*.” 2021 DOI:[10.1109/ICASSP39728.2021.9413520](https://doi.org/10.1109/ICASSP39728.2021.9413520)

Plagiarism Report

Attached plagiarism report here that can be obtained from the library.

Report Approval Certificate

The report of the project, “Title” has been approved based on the following evaluation guideline.

Project Evaluation Guidelines

Artifacts Guidelines	
Analysis and Design artifacts are syntactically correct (use-case model, SSDs, domain model, class diagram, SDs, ERDs, Flow charts, Activity Diagram, DFDs)	
Consistency and traceability have been maintained among different artifacts	
General Guidelines	
Formatting (font style, indentation) is according to the FYP template and consistent throughout the document	
Captions are added to all the figures and tables. Figure captions must be placed below each figure, and table captions must be provided above the table	
Each figure or table is followed by some text describing what it represents	

Name & Signature
(Examiner3)

Appendix

User Manual of the software